Bogdanovic Stefan, Rodari Simone,
Pignataro Gabriele

POLITECNICO
MILANO 1863

Students&Companies
Design Document

Deliverable:	DD
Title:	Students&Companies
Authors:	Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele
Version:	1.0
Date:	07-January-2025
Download page:	https://github.com/SimoPolimi/RodariPignataroBogdanovic
Copyright:	Copyright © 2025, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

Contents

Table of Contents 3

List of Figures 5

List of Tables 5

1 Introduction 6
- 1.1 Purpose 6
- 1.2 Scope 6
- 1.3 Definitions, Acronyms, Abbreviations 6
  - 1.3.1 Definitions 6
  - 1.3.2 Acronyms 6
  - 1.3.3 Abbreviations 6
- 1.4 Revision History 7
- 1.5 Reference Documents 7
- 1.6 Document Structure 7

2 Architectural Design 8
- 2.1 Overview 8
- 2.2 Component view 8
- 2.3 Deployment view 10
- 2.4 Runtime View 11
- 2.5 Component Interfaces 25
  - 2.5.1 WebAPP 25
  - 2.5.2 SheerID 26
  - 2.5.3 Database 26
  - 2.5.4 DAO 26
  - 2.5.5 AuthenticationManager 27
  - 2.5.6 AccountManager 27
  - 2.5.7 InternshipManager 27
  - 2.5.8 ContractManager 28
  - 2.5.9 FeedbackManager 28
  - 2.5.10 DiscussionManager 28
  - 2.5.11 InterviewManager 28
  - 2.5.12 InternshipAnalyzer 28
  - 2.5.13 CVAnalyzer 28
  - 2.5.14 AffinityAnalyzer 28
  - 2.5.15 NotificationManager 28
  - 2.5.16 MailService 28
- 2.6 Selected Architectural Styles and Patterns 29
  - 2.6.1 3-tier Architecture 29
  - 2.6.2 Model View Controller Patter 29
  - 2.6.3 RESTful API 29
- 2.7 Other Design Decisions 29
  - 2.7.1 Load Balancer 29

3 User Interface Design 30

4 Requirements Traceability 34

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

5 Implementation, Integration, and Test Plan **40**
5.1 Overview 40
5.2 Testing and Implementation Plan 40
5.3 Additional System Testing 40

6 Effort Spent **42**

References **43**

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

List of Figures

1 High level architecture 8
2 Component view of the system 9
3 Deployment view 10
4 Dependencies of the components 40

List of Tables

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

1 Introduction

1.1 Purpose

Internships are a vital bridge between academic learning and professional experience, providing students with opportunities to apply their skills and companies with access to emerging talent. However, finding the right match between students and internships can be a complex and inefficient process. The Students&Companies (S&C) platform addresses these challenges by providing an integrated system that connects university students with companies offering internships. S&C provides a wide range of features offering a seamless experience for students from the search of an internship to its completion and for companies an easy way to advertise their internship projects. The platform also lets universities to monitor and support their students throughout their internship, overseeing its execution and supporting the involved parties, should problems arise.

1.2 Scope

The Students&Companies platform provides its services to different users, identifiable into three main classes: students, companies and universities. Each one of them has different roles and interacts with the system in various ways. S&C is aimed mainly at university students that want to take part in an internship program, as they can search for new internships suiting their preferences or let the system inform them when internships matching their profile are available. S&C is also able to provide suggestions on how to improve their profile to make it more appealing to companies. The platform also supports companies easing the internships' management. In doing so S&C can be used to create questionnaires to select the most suitable candidates for an internship. Similarly to the suggestions for students, the system can analyze the internship offer to identify eventual improvements that can be made. Hopefully, every internship comes to an end without problems but if that is not the case, interns and companies can communicate through the platform possibly involving the intern's alma mater as a supervisor which can also terminate the internship if the parties cannot resolve the controversy.

1.3 Definitions, Acronyms, Abbreviations

1.3.1 Definitions

- **Recommendations**: Process in which the system informs students when an internship that might interest them becomes available and can inform companies about the availability of student CVs corresponding to their needs.
- **SheerID**[1]: Platform that offers an API to verify students by using their academic credentials.
- **Contract**: The contract represents the lifecycle of an internship application, from initiation to its conclusion.

1.3.2 Acronyms

- **S&C**: Students & Companies
- **DAO**: Data Accessing Object
- **API**: Application Programming Interface
- **REST**: REpresentational State Transfer

1.3.3 Abbreviations

- **R***: functional requirements.

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

1.4 Revision History

- 07/01/2025 - Version 1.0.

1.5 Reference Documents

- Assignment document “Assignment RDD AY 2024-2025.

1.6 Document Structure

Introduction: Provides a brief description of the project, focusing on the reasons behind its development and the goals to be achieved, as previously explained in the RASD. Architectural Design: This section describes the architecture at different levels of granularity.
User Interface Design: This part presents the wireframes that represent the platform’s user interface.
Requirements Traceability: Explains how the goals defined in the previous document are fulfilled through the interaction between the requirements and the components described in this document.
Implementation, Integration, and Test Plan: Provides details about the implementation, integration, and testing processes, useful for developers and platform producers.
Effort Spent: Shows the effort involved in writing this document.
Bibliography: Contains references to any documents and software used to write this paper.

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

2 Architectural Design

2.1 Overview

This section is intended to explain in detail all the components used by the system in order to offer all the functionalities.
The figure 1 below shows the high-level architecture of the system-to-be. All the components are better explained in the following paragraphs.

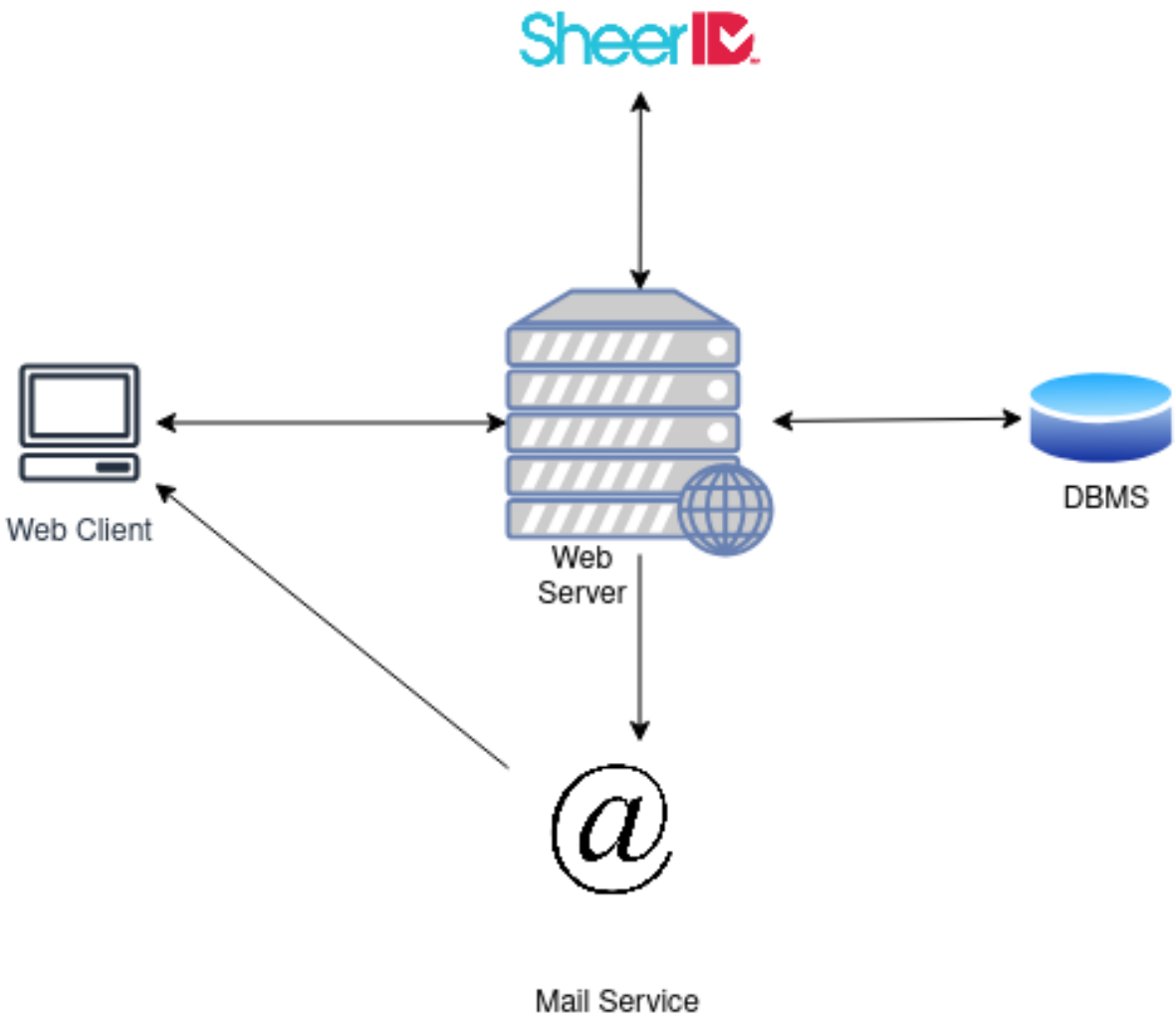


Figure 1: High level architecture

2.2 Component view

The Students&Companies platform will be implemented using a three tier architecture. Figure 2 shows the component view diagram of the platform. The WebApp (presentation layer) will use functions implemented in the business layer through API calls. The business layer interacts with SheerID for the student login functionality.

The system is comprised of the following components:

- **Data Accessing Object (DAO)**: a set of classes utilized to interact with the database, facilitating

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

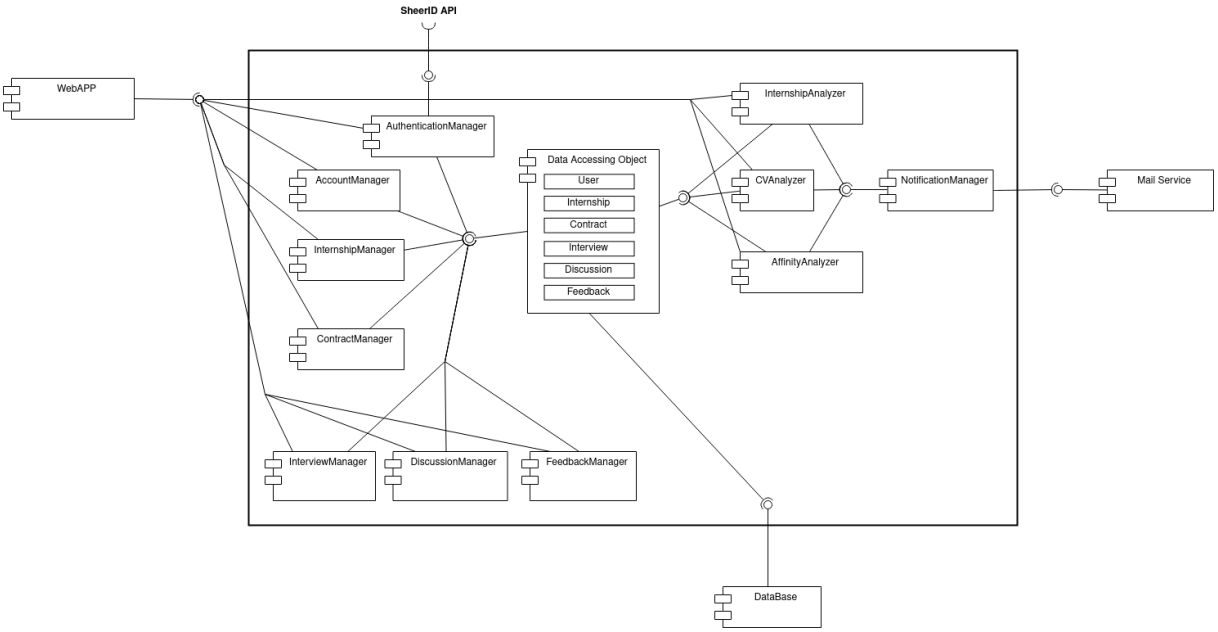


Figure 2: Component view of the system

the retrieval and uploading of information. This approach ensures a single component manages the database connection and handles the query language.

- **AuthenticationManager**: component used for managing user's registration and login. It uses DAO's interfaces to manage companies and universities information and SheerID's API to manage students' login.
- **AccountManager**: component used to allow the user to modify his profile.
- **InternshipManager**: component used to help users to manage internships.
- **ContractManager**: component used to help users to manage contract stipulations.
- **InterviewManager**: component used to help users to manage interviews.
- **DiscussionManager**: component used to help users to manage discussions.
- **FeedbackManager**: component used to help users to manage feedback.
- **InternshipAnalyzer**: component that produces analytics on internship descriptions based on info collected from the database through DAOs. It relies on the NotificationManager to send notifications to final users.
- **CVAnalyzer**: component that produces analytics on CVs based on info collected from the database through DAOs. It relies on the NotificationManager to send notifications to final users.
- **AffinityAnalyzer**: component that produces analytics on CVs and internship positions based on info collected from the database through DAOs. It relies on the NotificationManager to send notifications to final users.
- **NotificationManager**: component used to abstract the service used to send notifications from the other components.

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

2.3 Deployment view

The organization of S&C's physical infrastructure is shown in figure 3. As it is possible to see, the system uses a single server to host the database, while an array of replicated servers runs the components related to the buisness logic.

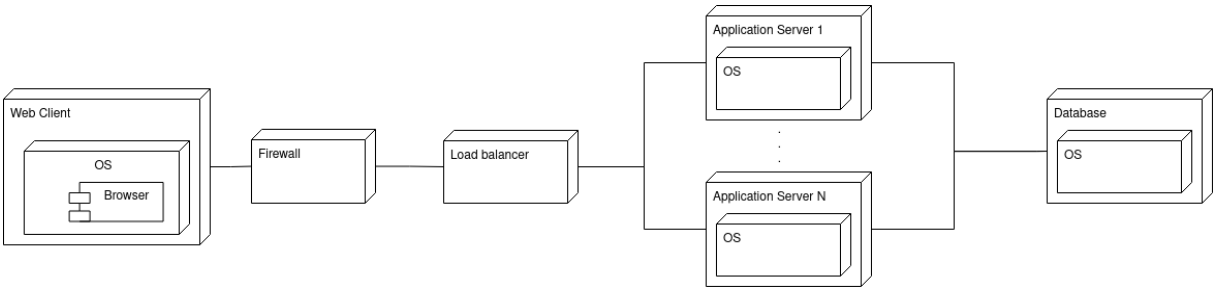


Figure 3: Deployment view

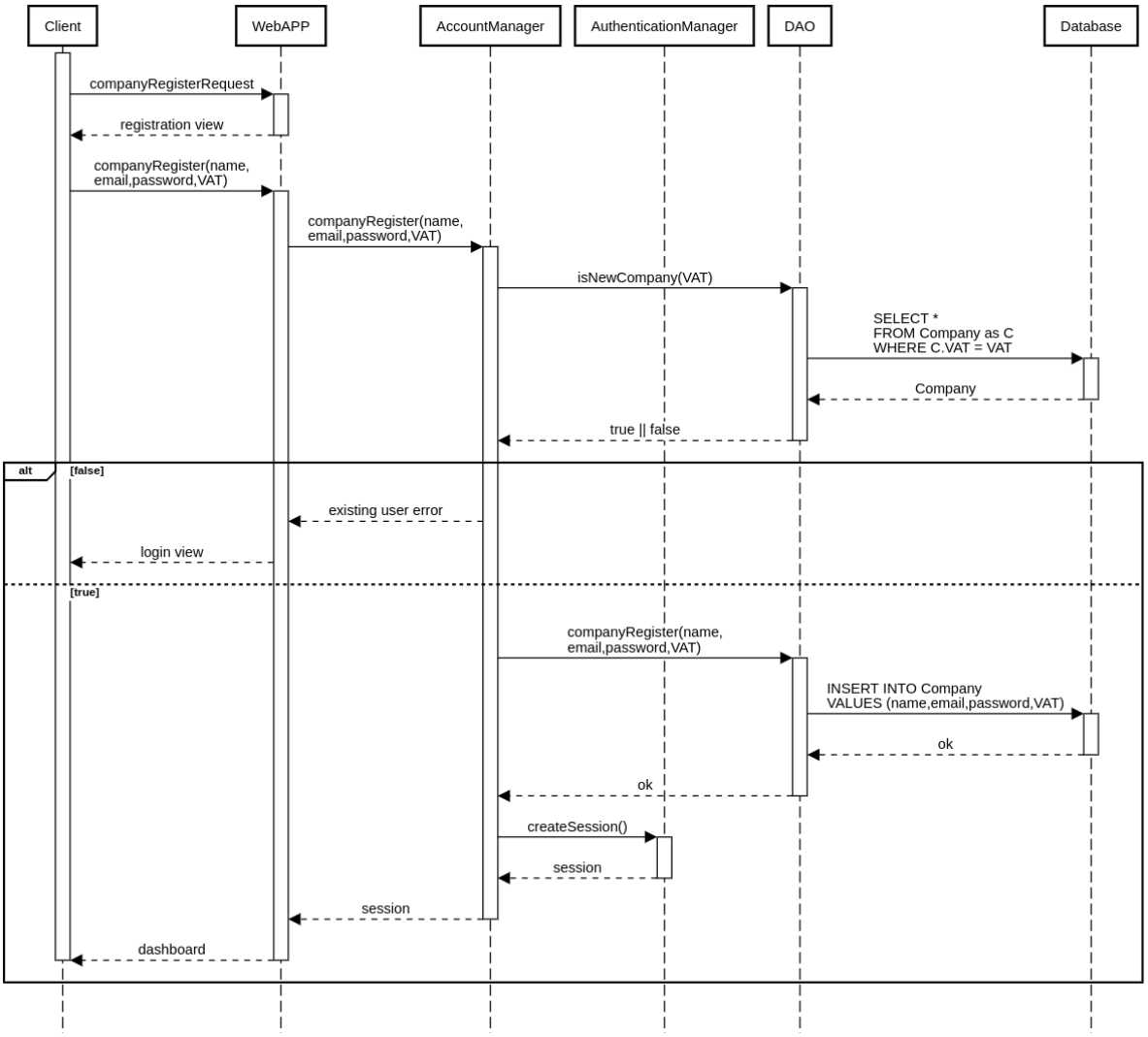
In more detail, the system architecture is made of:

- **WebClient**: the user's device, running a browser which is used to interact with the system.
- **Firewall**: a firewall to protect the system from potential malicious attacks.
- **Load balancer**: a component used to distribute incoming requests to different replicated servers based on individual loads.
- **Application server**: the server in charge of handling requests from the clients. To ensure acceptable performance even under high loads this server is replicated.
- **Database**: a server dedicated to run the DBMS.

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

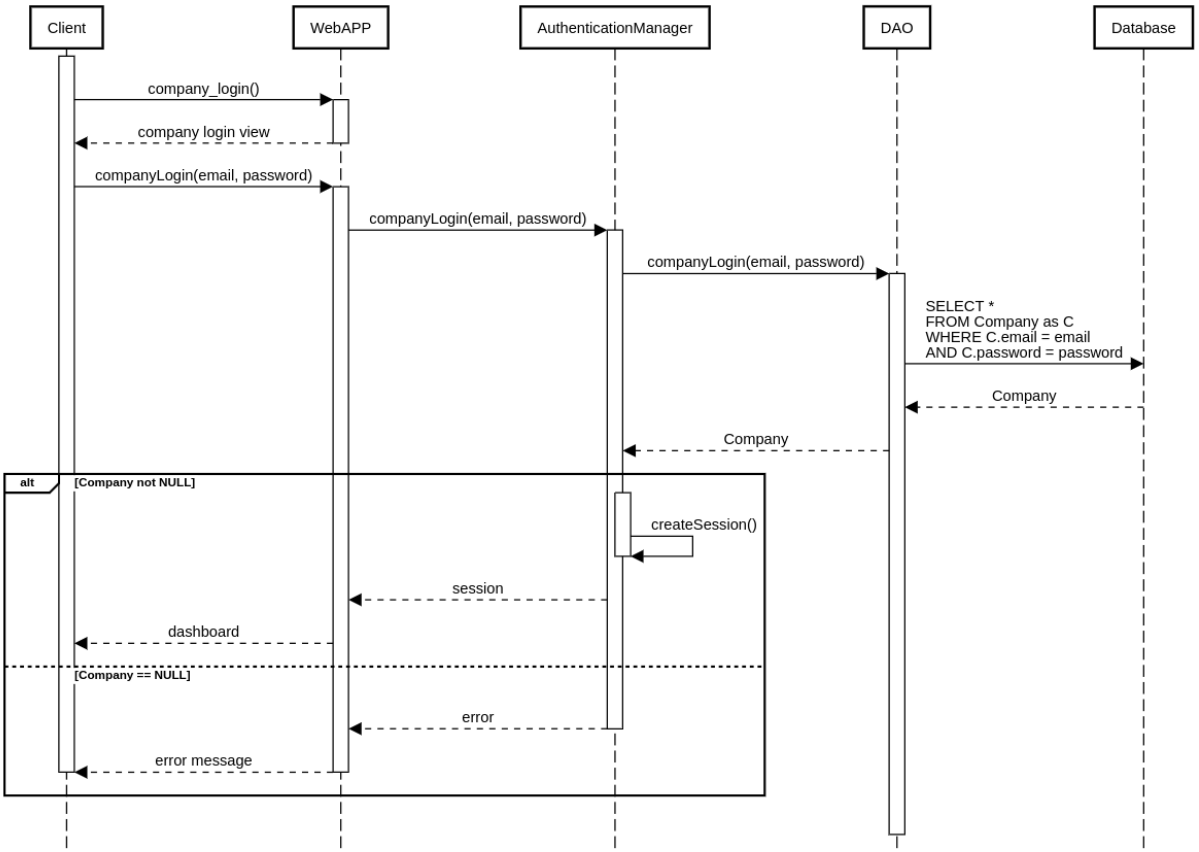
2.4 Runtime View

[UC1] - Company Registration

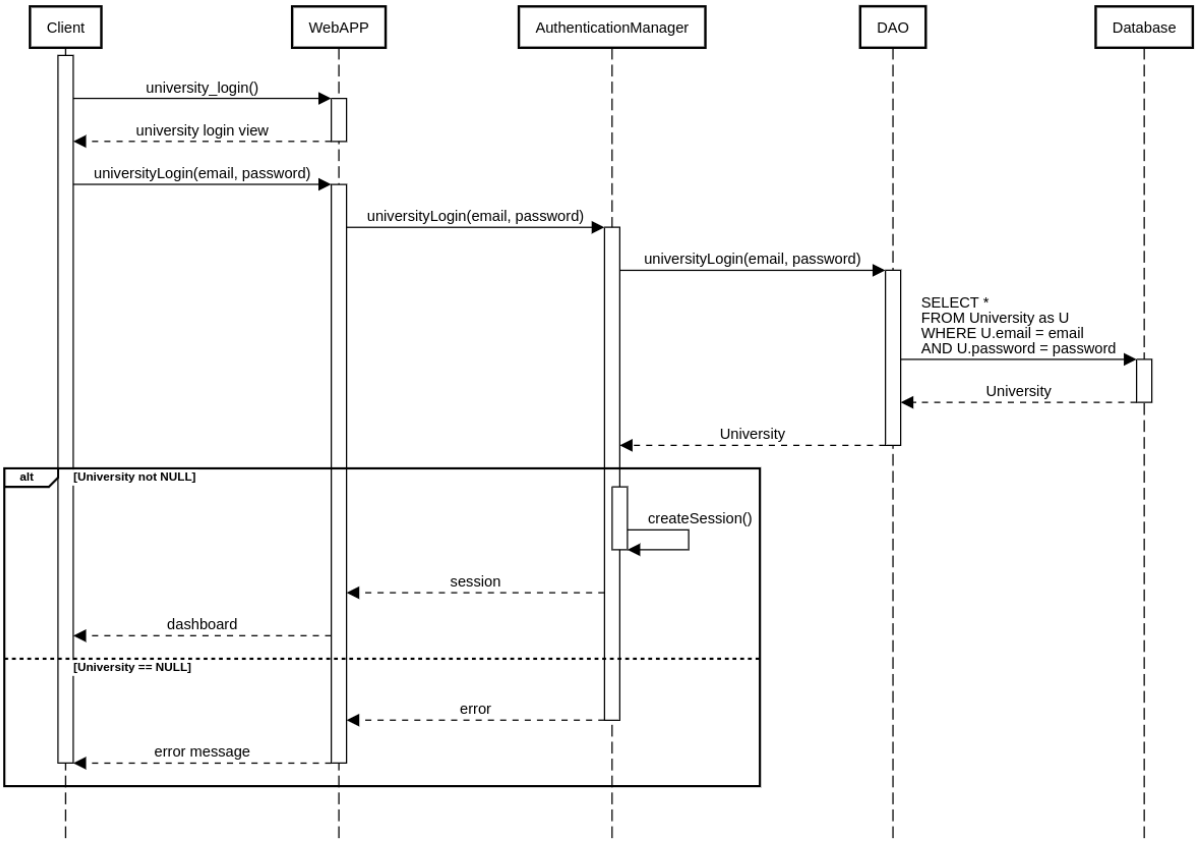


Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC2] - Company Login

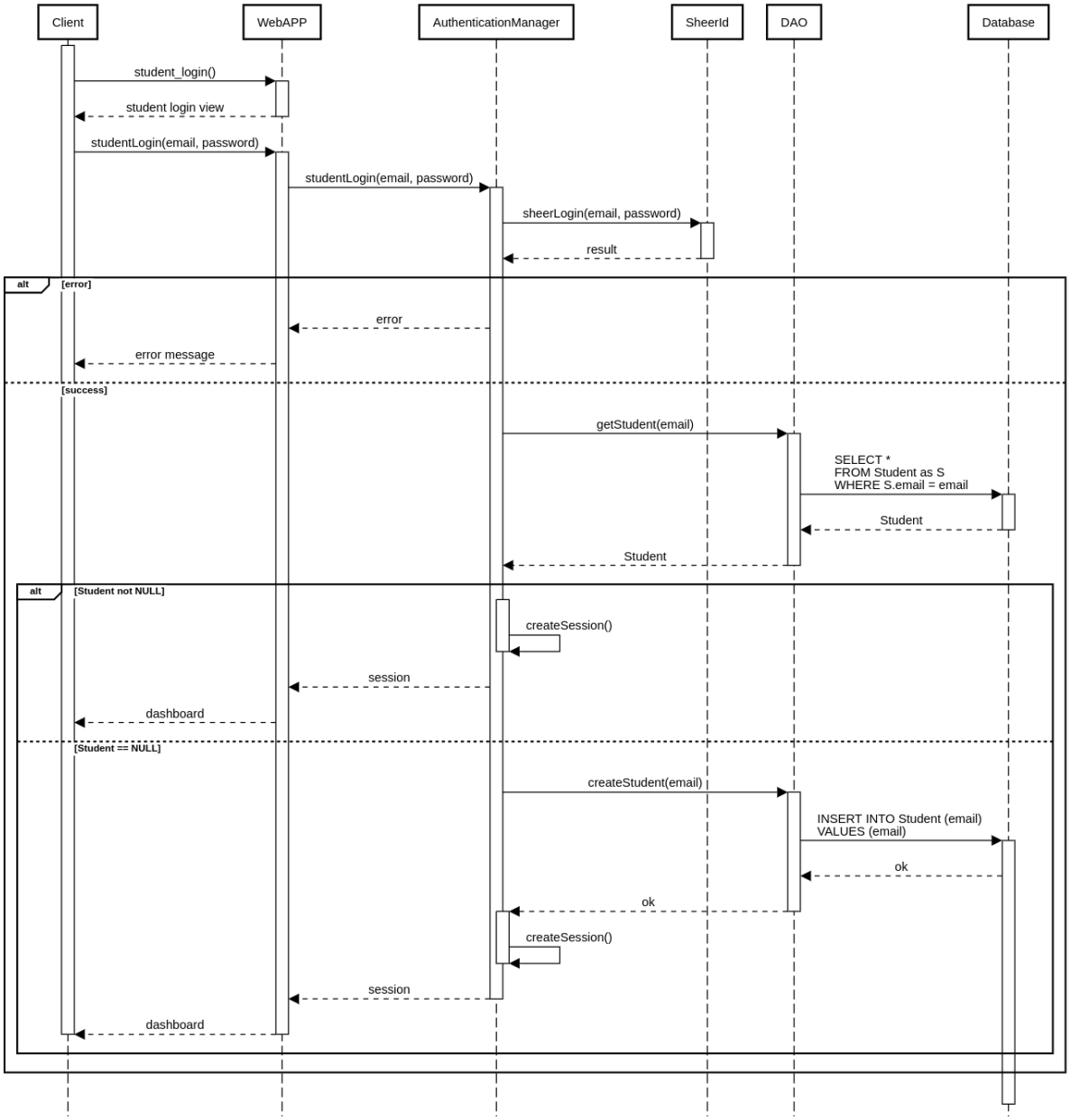


[UC3] - University Login



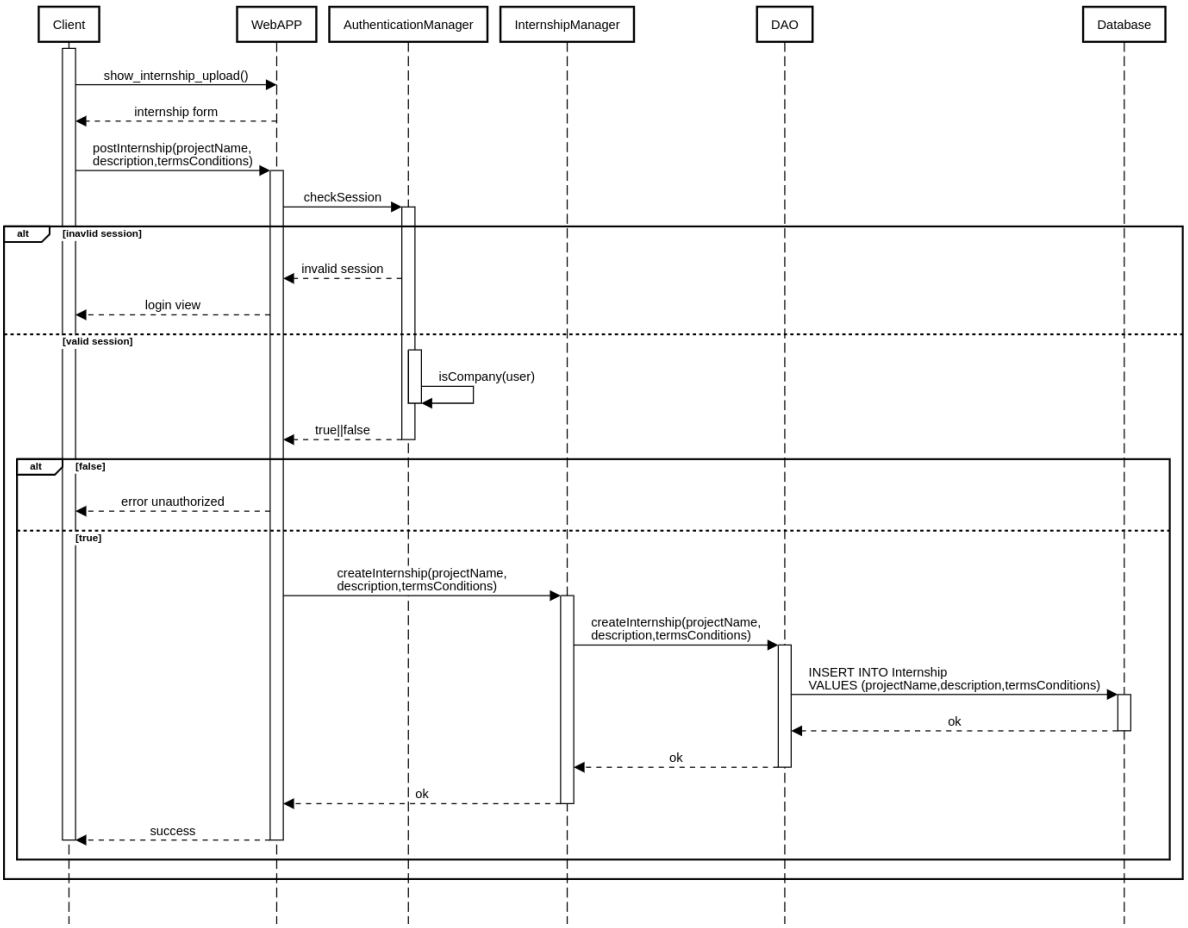
Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC4] - Student Login



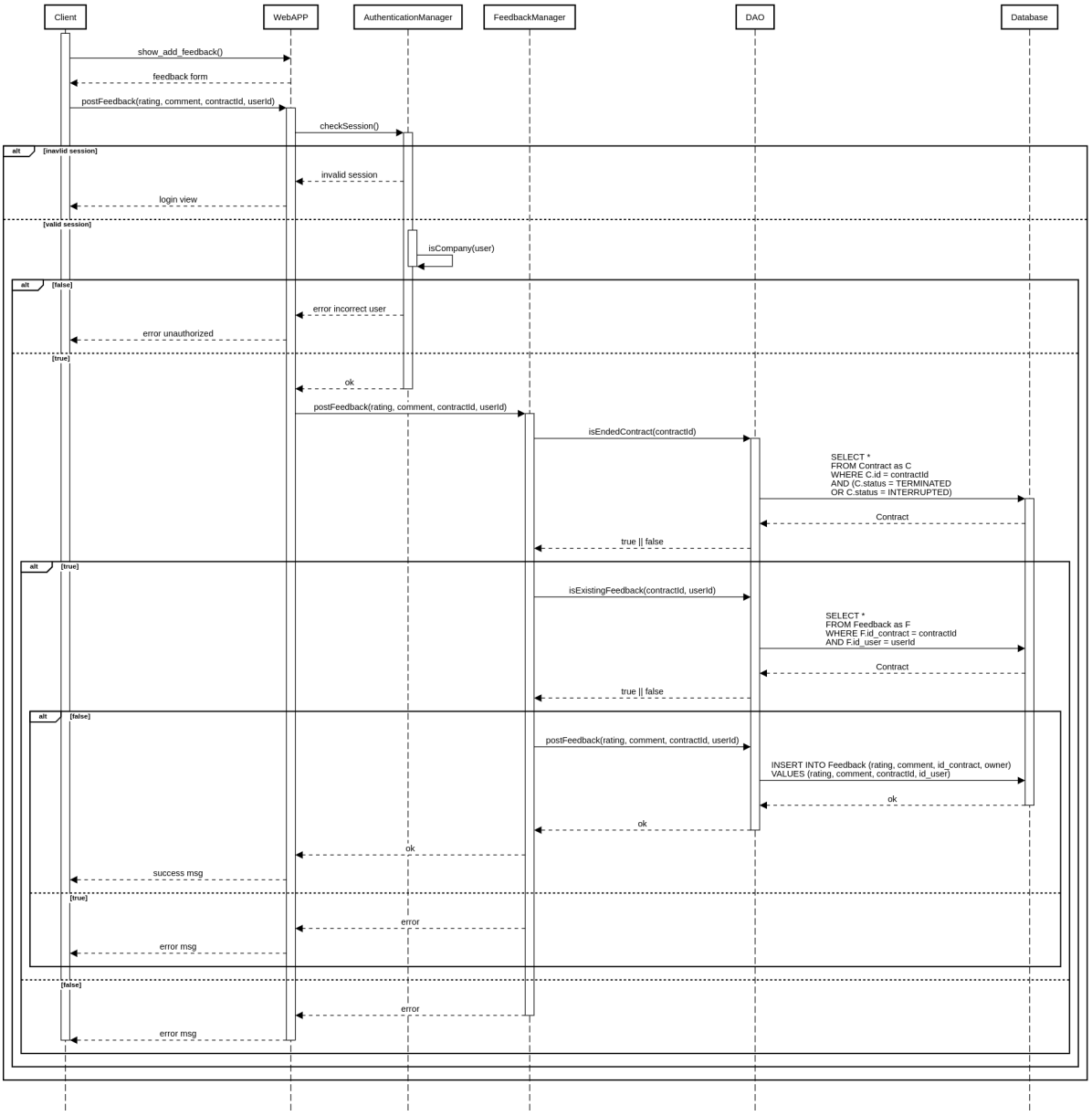
Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC5] - Post Internship



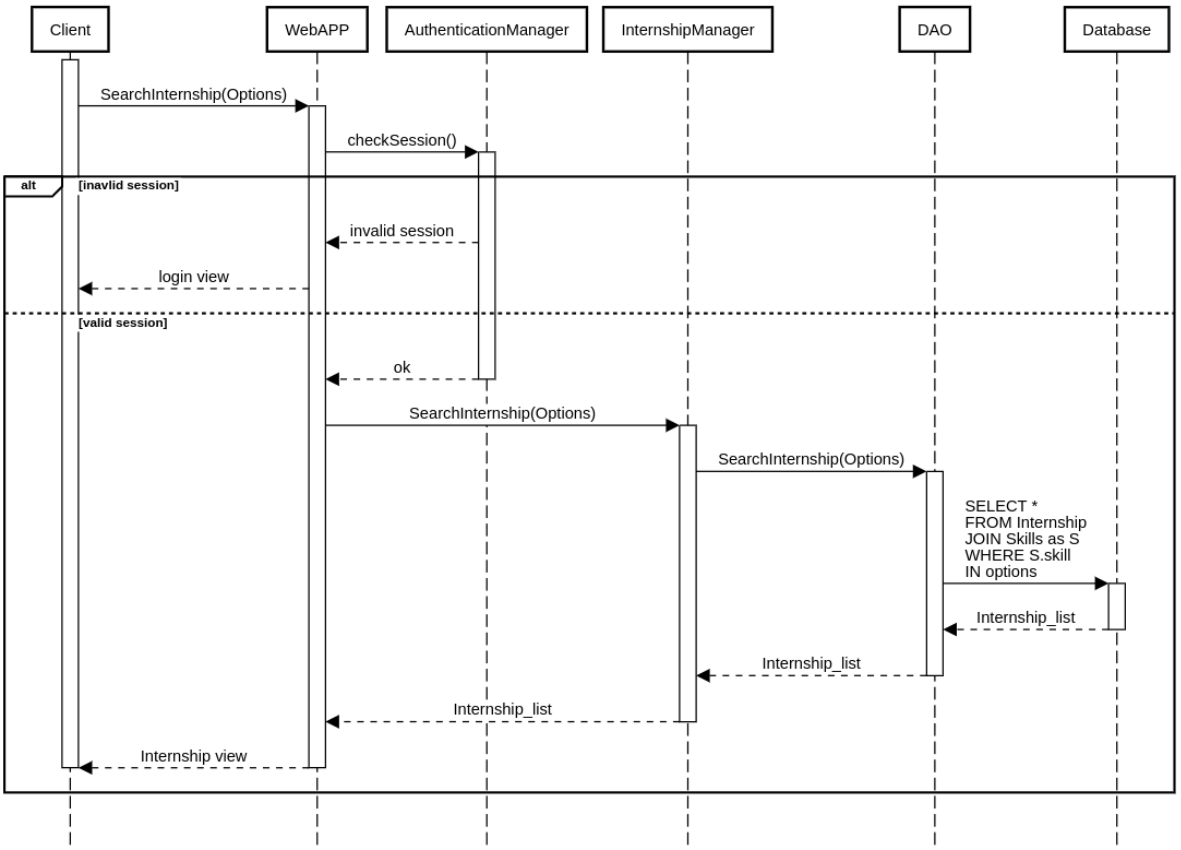
Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC6] - Post Feedback



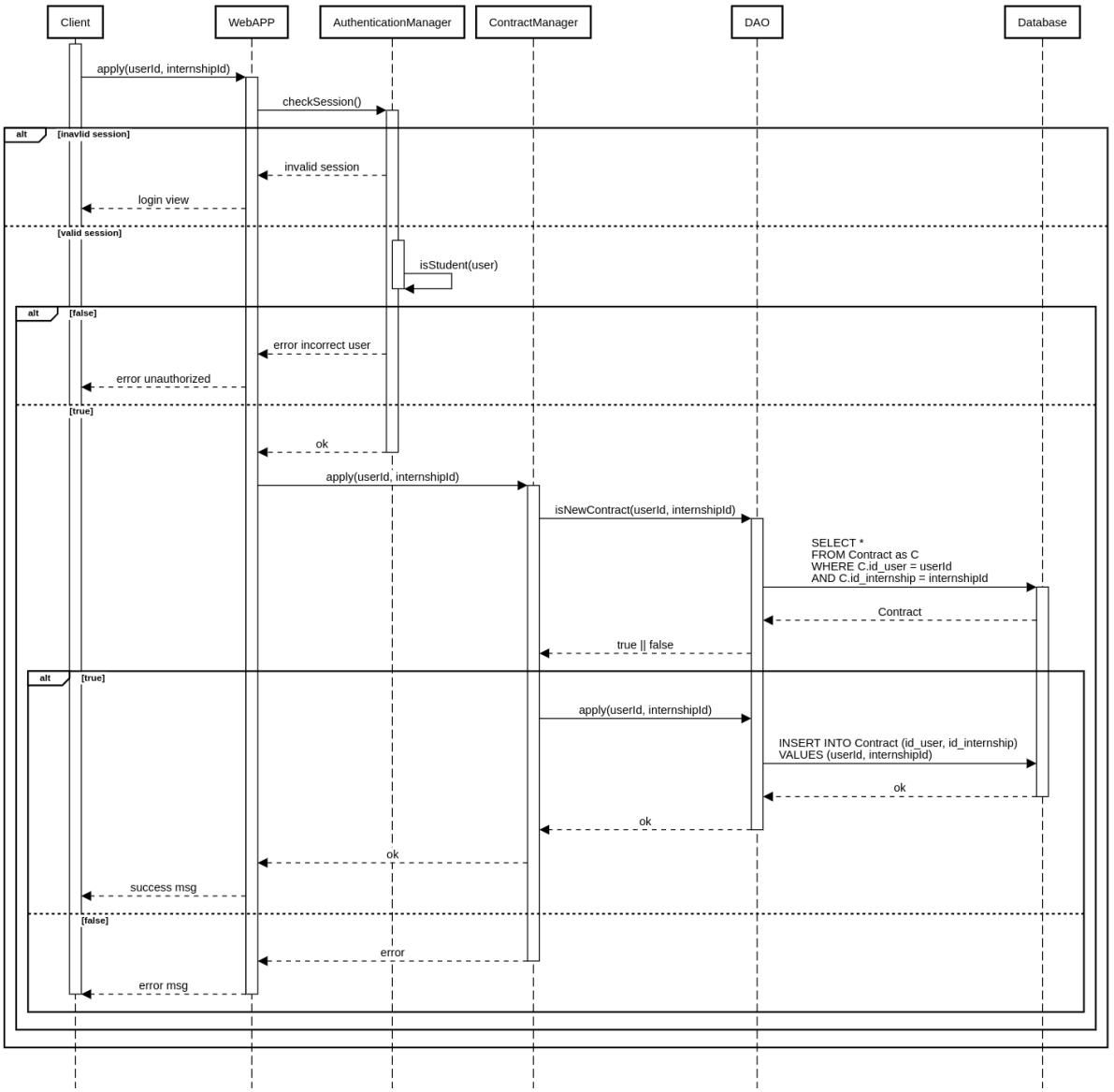
Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC7] - Search Internship



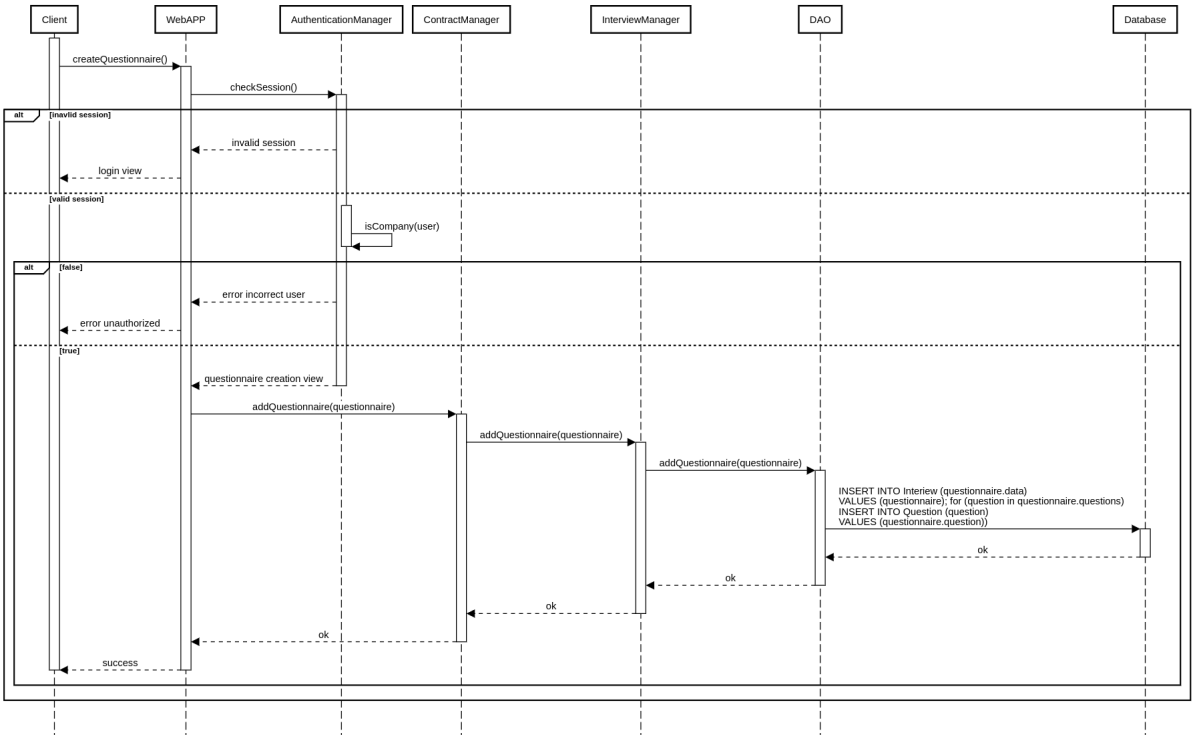
Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC8] - Apply for an Internship



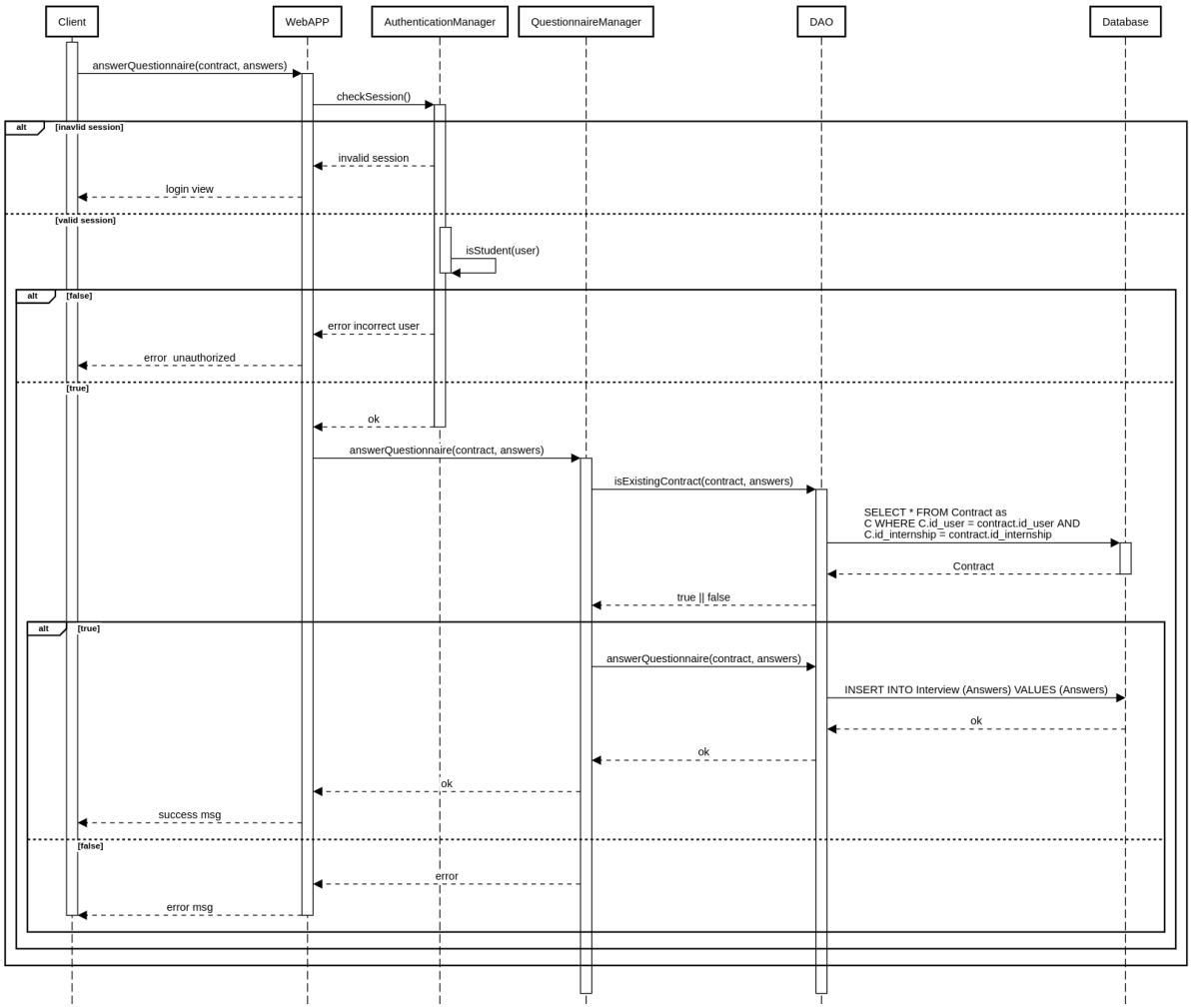
Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC9] - Questionnaire Creation



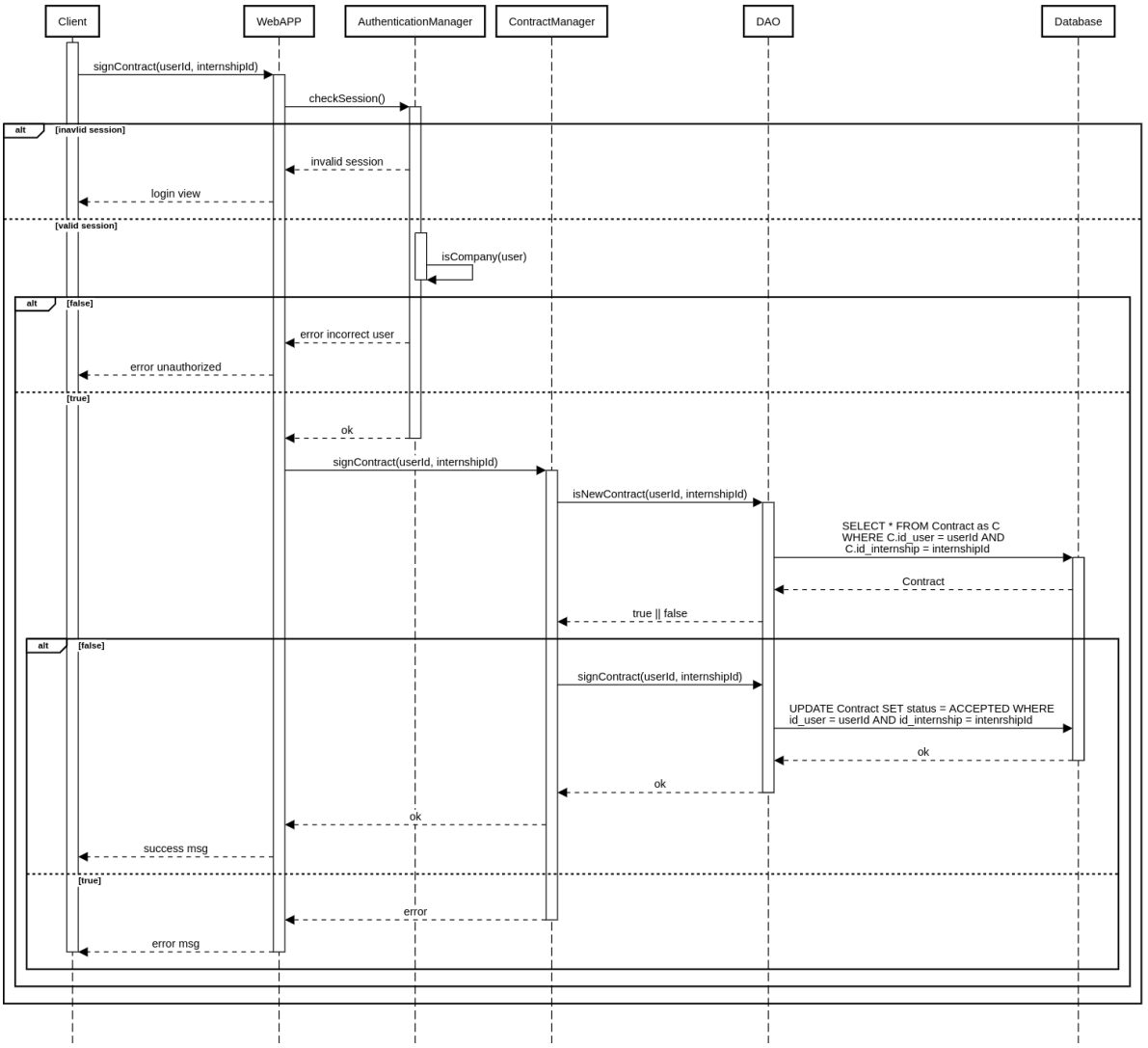
Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC10] - Answer Questionnaire Interview



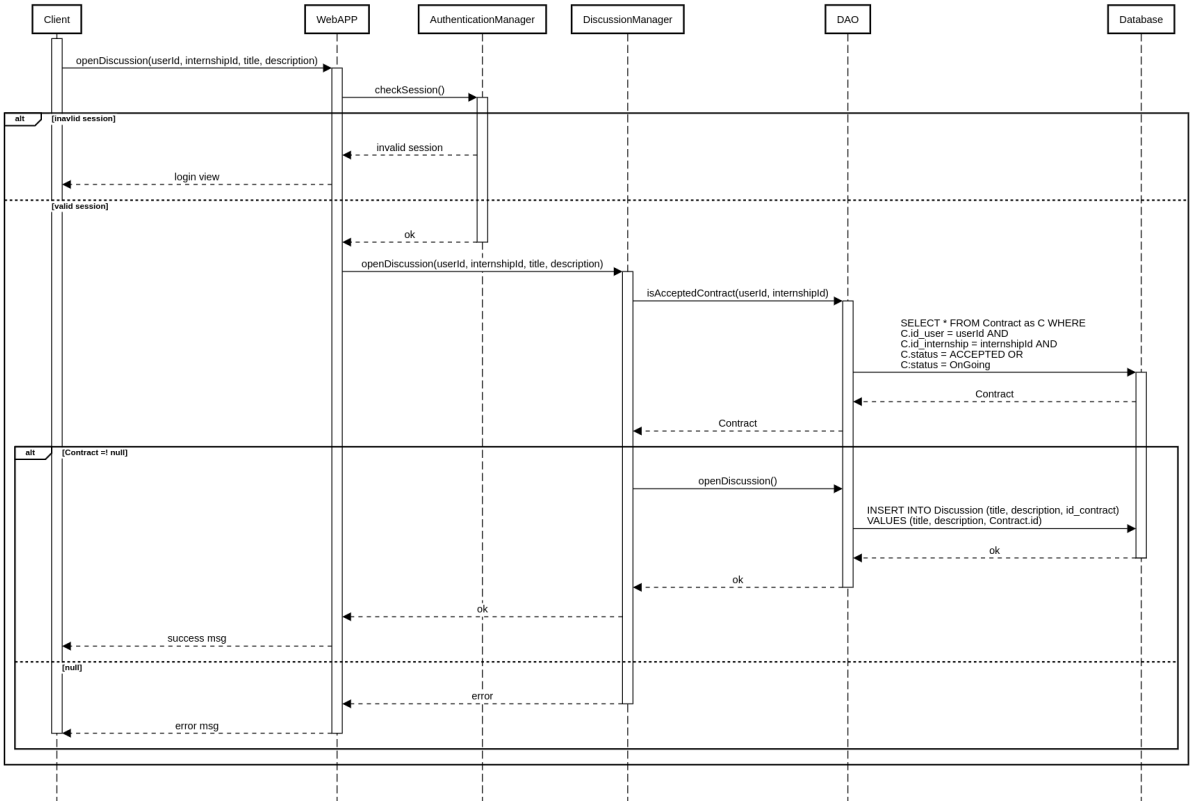
Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC11] - Sign Contract

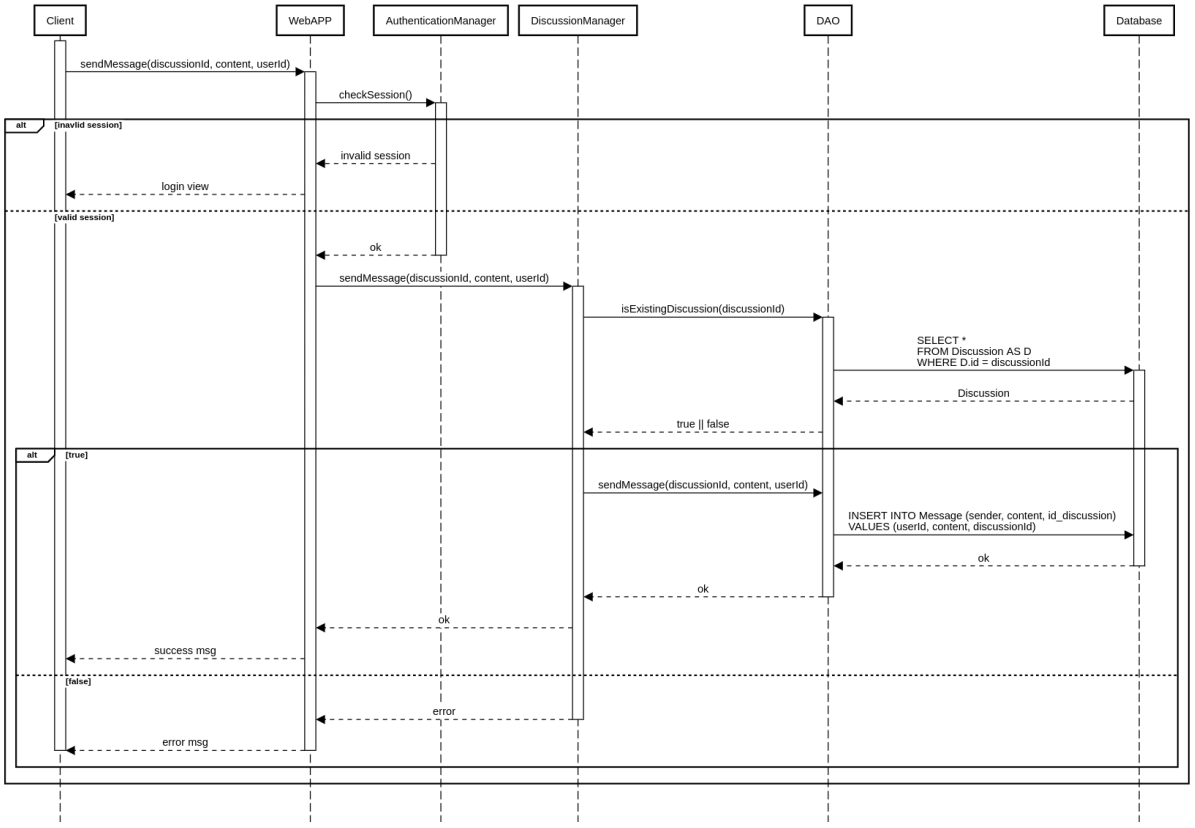


Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC12] - Open Discussion

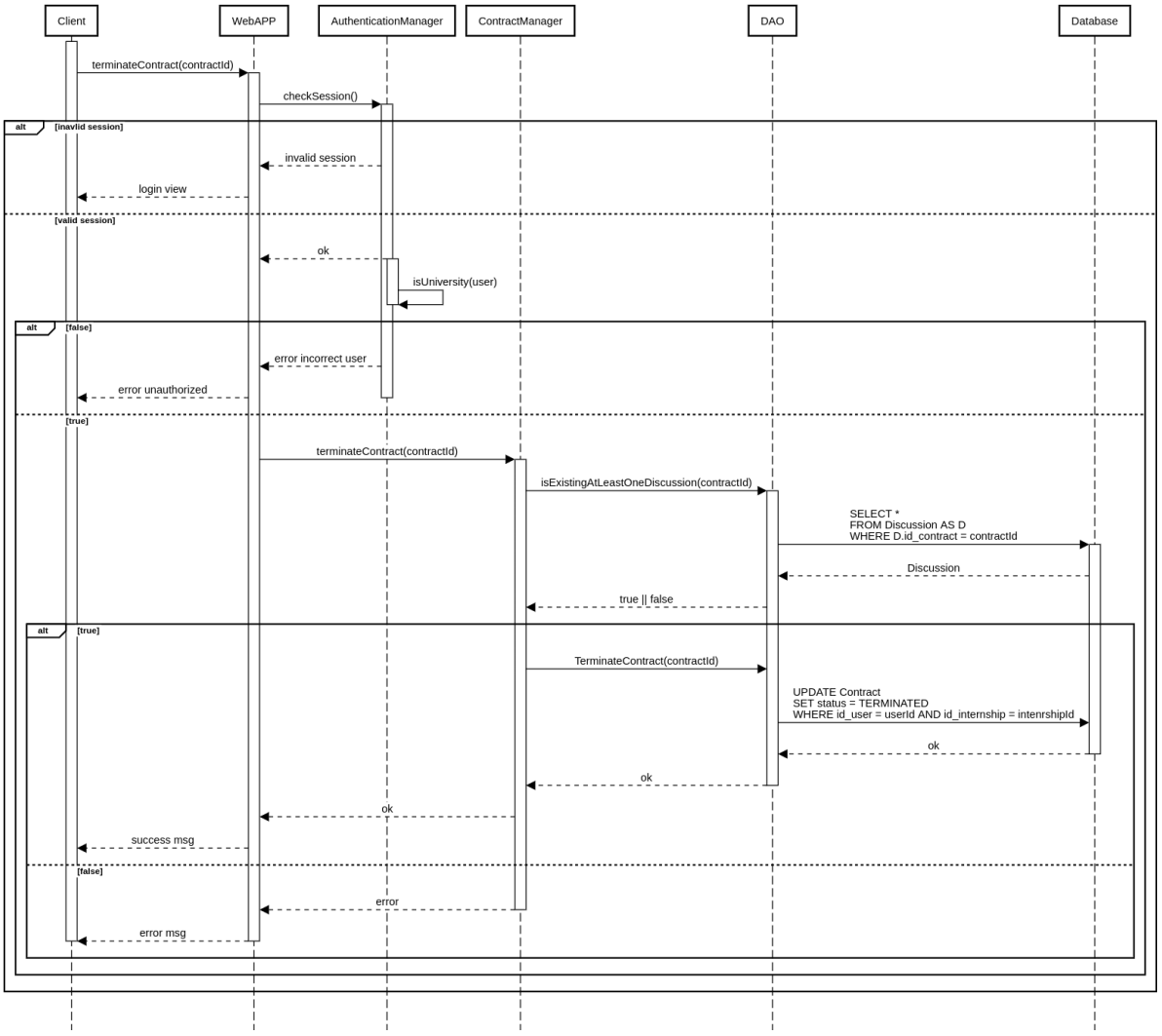


[UC13] - Send Message in a Discussion

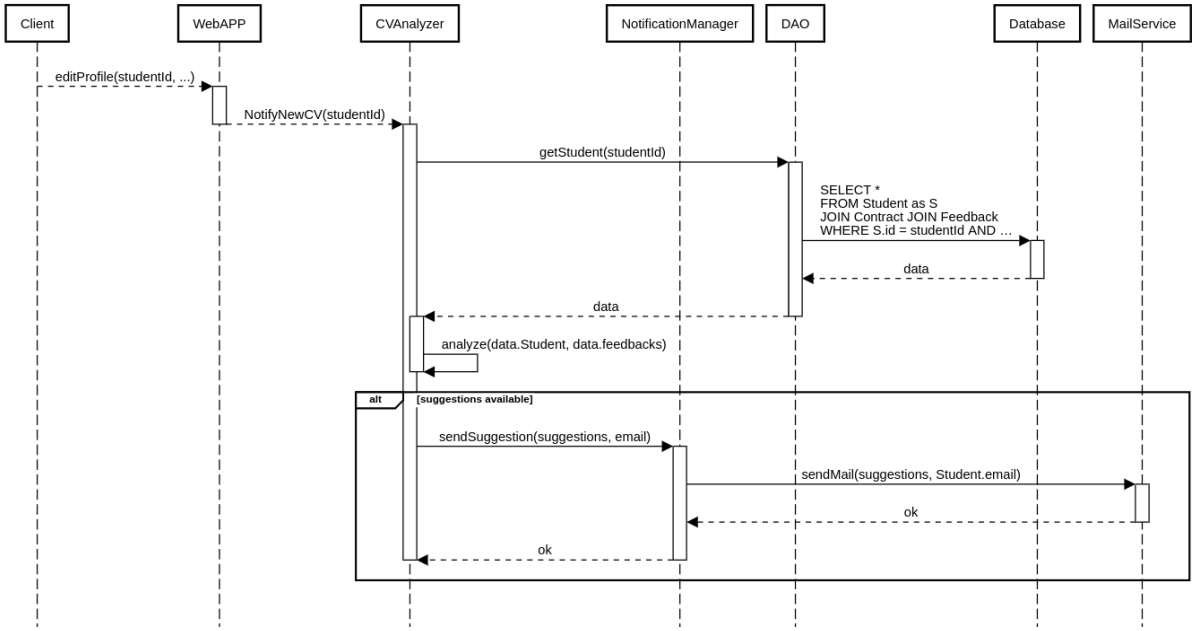


Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC14] - Terminate Internship

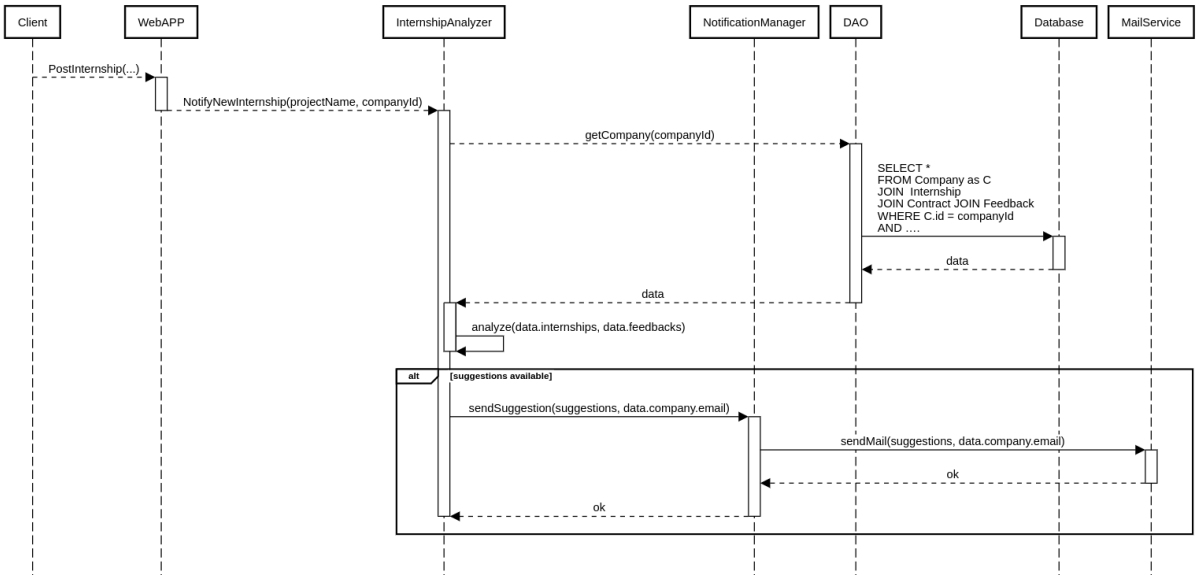


[UC15] - Generate CV Improvement Suggestion

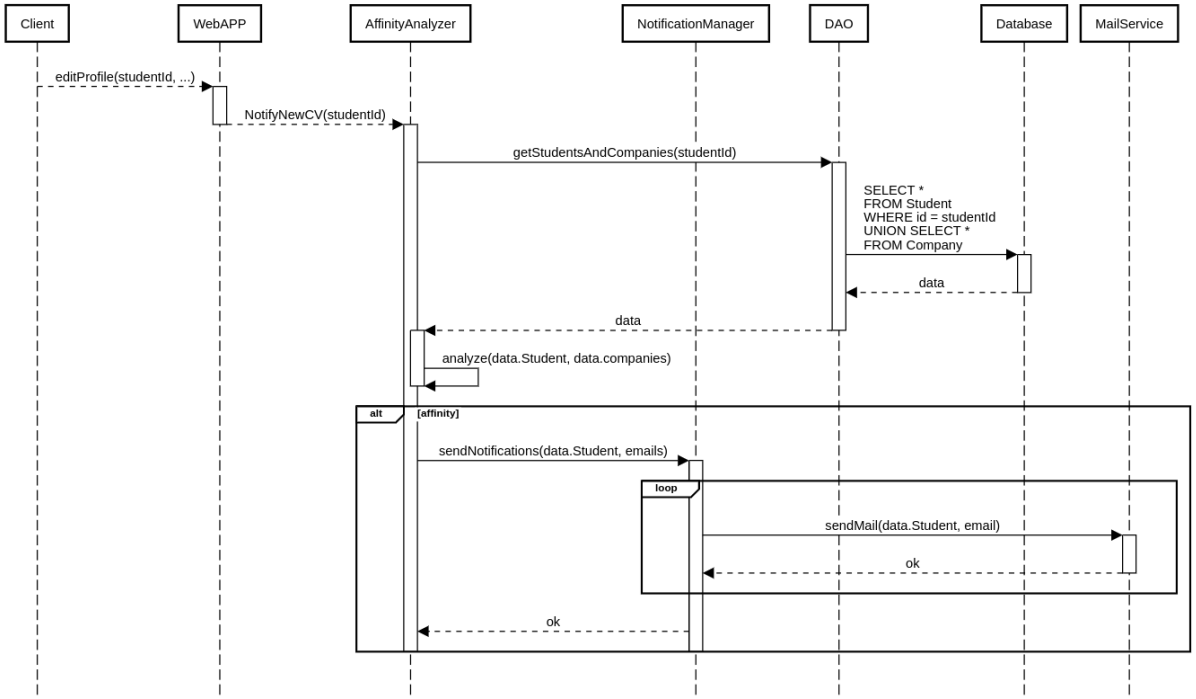


Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC16] - Generate Internship Improvement Suggestion

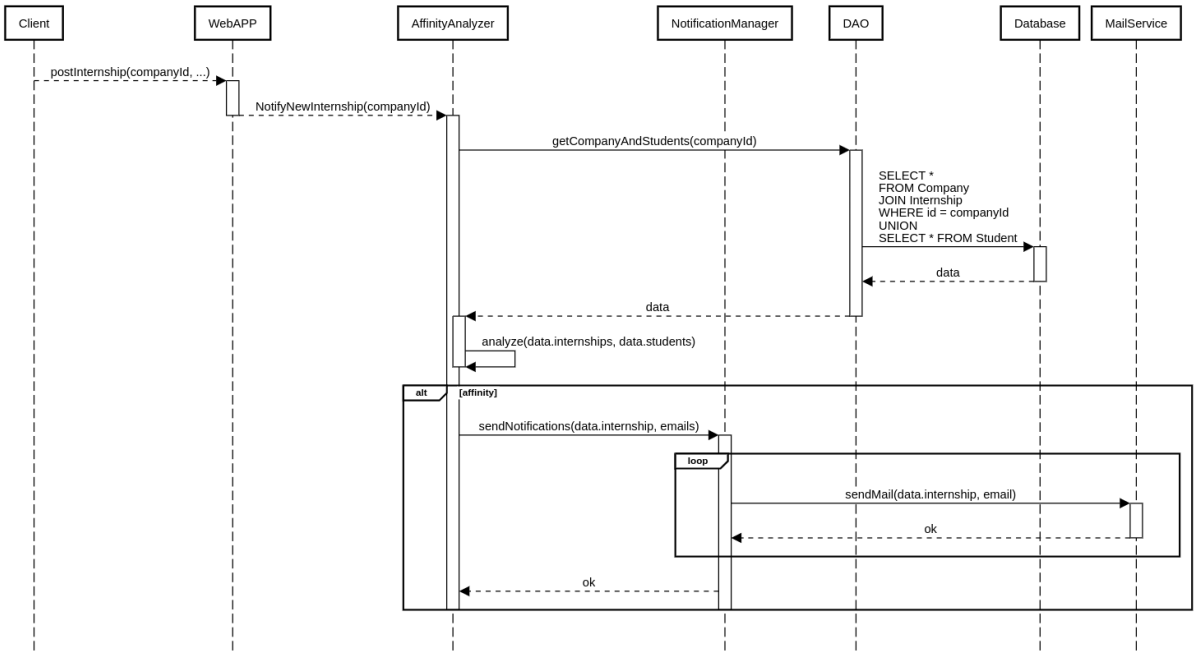


[UC17] - Student Recommendation

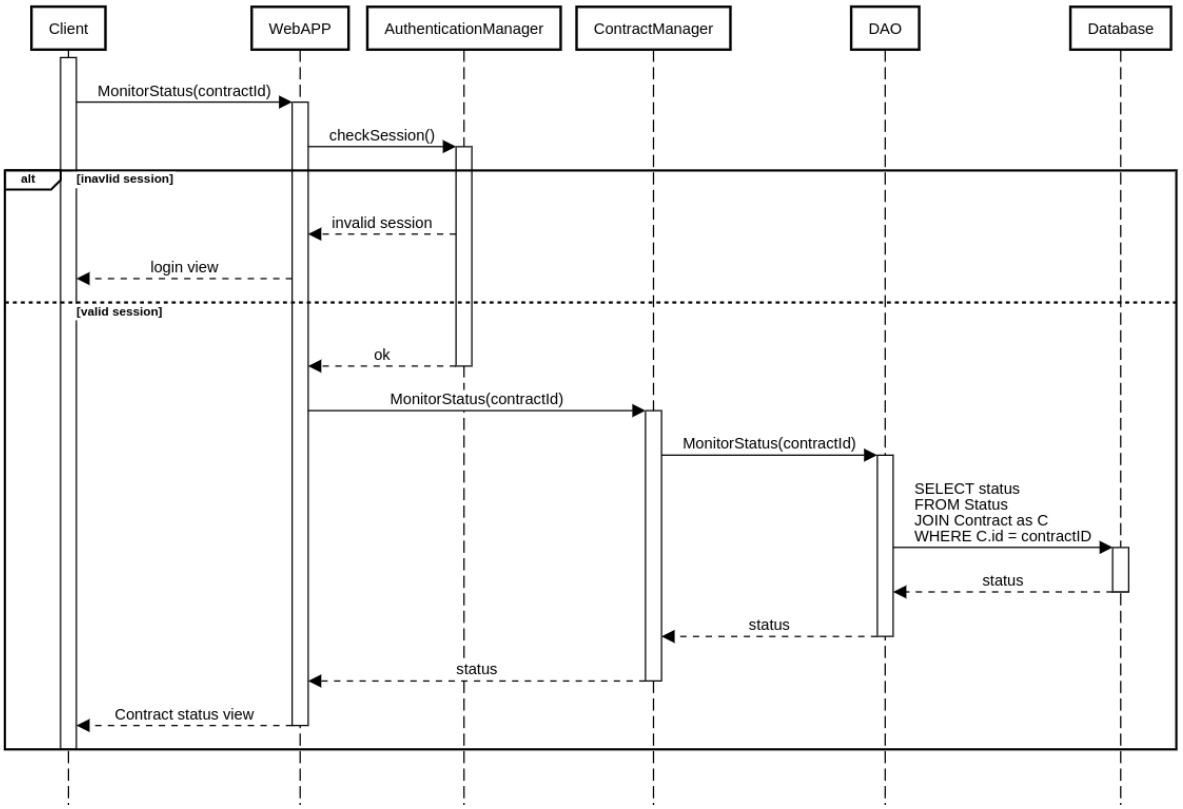


Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC18] - Internship Recommendation

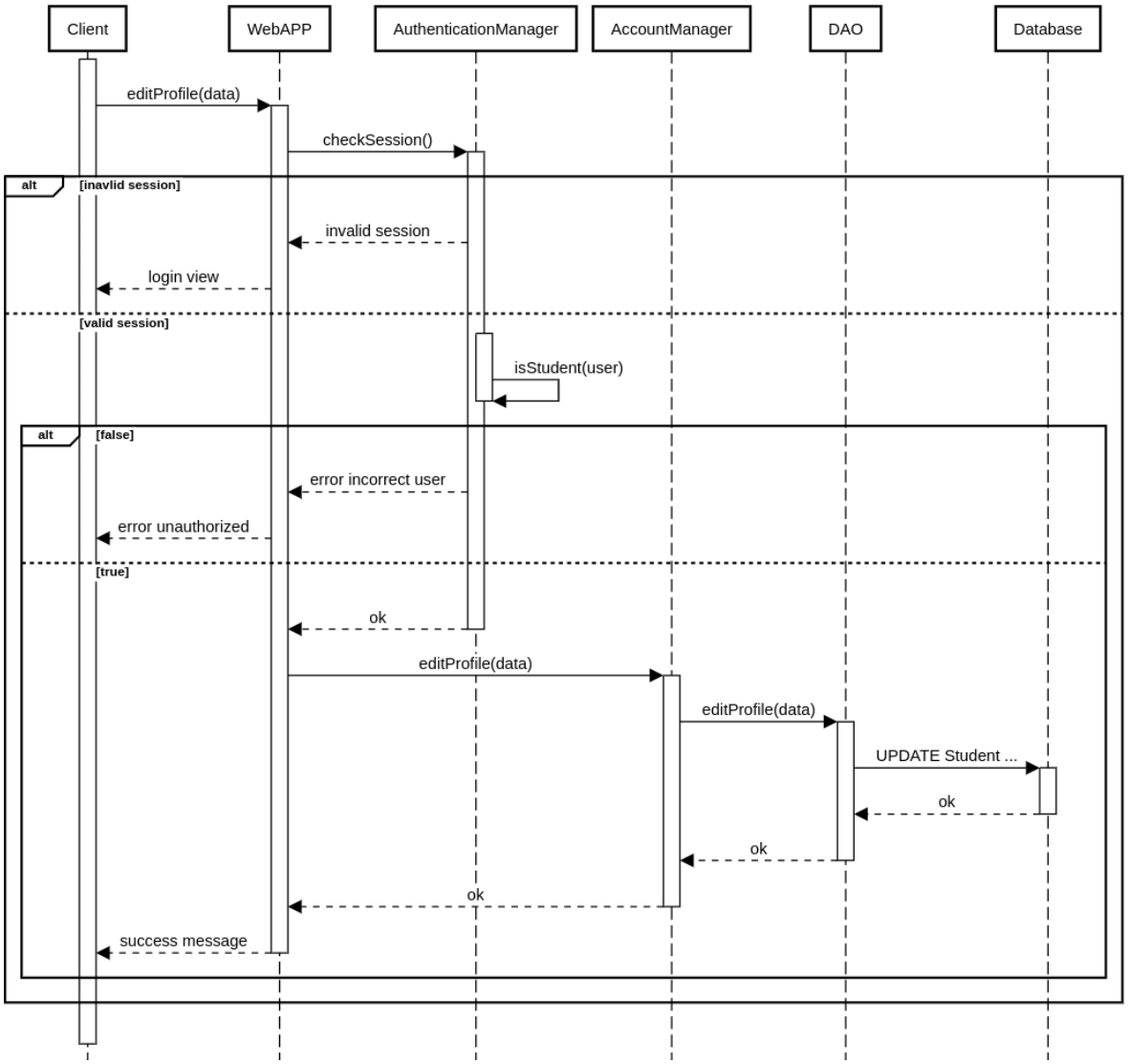


[UC19] - Monitor Internship Status



Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

[UC20] - Edit Profile



2.5 Component Interfaces

2.5.1 WebAPP

- void: companyRegister.onClick()
- void: companyRegister(name, email, password, VAT)
- void: companyLogin.onClick()
- void: companyLogin(username, password)
- void: universityLogin.onClick()
- void: universityLogin(email, password)
- void: studentLogin.onClick()
- void: studentLogin(email, password)

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

- void: postInternship(projectName, description, termsConditions)
- void: postFeedback(rating, comment, contractId, userId)
- void: searchInternship(Options)
- void: apply(userId, internshipId)
- void: createQuestionnaire()
- void: answerQuestionnaire(contract, answers)
- void: signContract(userId, internshipId)
- void: openDiscussion(userId, internshipId, title, description)
- void: sendMessage(discussionId, content, userId)
- void: terminateContract(contractId)
- void: createQuestionnaire()
- void: monitorStatus(contractId)

2.5.2 SheerID

- bool: SheerLogin(email, password)

2.5.3 Database

- SELECT ...
- INSERT ...
- UPDATE ...

2.5.4 DAO

- bool: isNewCompany(VAT)
- bool: isEndedContract(contractId)
- bool: isExistingFeedback(contractId, userId)
- bool: isExistingDiscussion(discussionId)
- bool: isAcceptedContract(userId, internshipId)
- bool: isExistingContract(contract, answers)
- bool: companyRegister(name, email, password, VAT)
- bool: companyLogin(username, password)
- bool: universityLogin(email, password)
- Student: getStudent(email)
- bool: createStudent(email, password)

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

- bool: postInternship(projectName, description, termsConditions)
- bool: postFeedback(rating, comment, contractId, userId)
- List<Internship>: SearchInternship(Options)
- bool: apply(userId, internshipId)
- bool: createQuestionnaire()
- bool: answerQuestionnaire(contract, answers)
- bool: signContract(userId, internshipId)
- bool: openDiscussion(userId, internshipId, title, description)
- bool: sendMessage(discussionId, content, userId)
- bool: terminateContract(contractId)
- bool: editProfile(data)
- bool: addQuestionnaire(questionnaire)
- STATUS: monitorStatus(contractId)

2.5.5 AuthenticationManager

- JSON: createSession()
- bool: checkSession()
- Company: companyLogin(email, password)
- University: universityLogin(email, password)
- Student: studentLogin(email, password)
- bool: isCompany(user)
- bool: isStudent(user)
- bool: isUniversity(user)

2.5.6 AccountManager

- bool: companyRegister(name, email, password, VAT)
- bool: editProfile(data)

2.5.7 InternshipManager

- bool: postInternship(projectName, description, termsConditions)
- List<Internship>: SearchInternship(Options)

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

2.5.8 ContractManager

- bool: apply(userId, internshipId)
- bool: addQuestionnaire(questionnaire)
- bool: signContract(userId, internshipId)
- bool: terminateContract(contractId)
- STATUS: MonitorStatus(contractId)

2.5.9 FeedbackManager

- bool: postFeedback(rating, comment, contractId, userId)

2.5.10 DiscussionManager

- bool: openDiscussion(userId, internshipId, title, description)
- bool: sendMessage(discussionId, content, userId)

2.5.11 InterviewManager

- bool: answerQuestionnaire(contract, answers)

2.5.12 InternshipAnalyzer

- void: NotifyNewIntership(projectName, companyId)
- bool, String, email: analyze(internship, feedbacks)

2.5.13 CVAnalyzer

- void: NotifyNewCV(studentId)
- bool, String, email: analyze(student, feedbacks)

2.5.14 AffinityAnalyzer

- bool, String, List<email>: analyze(student, companies)
- bool, String, List<email>: analyze(internship, students)

2.5.15 NotificationManager

- bool: sendSuggestio(suggestions, email)
- bool: sendNotification(data, email)

2.5.16 MailService

- bool: sendMail(data, mail)

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

2.6 Selected Architectural Styles and Patterns

2.6.1 3-tier Architecture

The architecture chosen for the S&C system is a 3-tier client-server model, with the tiers assigned to the presentation, logic, and data layers, respectively. This architecture, characterized by highly decoupled components, enables the creation of a system that is both easily maintainable and flexible, allowing different aspects to be tested and updated independently. The tiers are described in more detail below:

- **Web Client**: This tier is represented by web browsers, which provide the user interface for interacting with the system.
- **Application Server**: This tier handles all business logic, processes requests from the web client, and generates the pages or responses to be displayed on the clients.
- **Database Server**: This layer is simply a database for storing persistently and providing data as needed.

2.6.2 Model View Controller Patter

The Model-View-Controller (MVC) pattern is a widely used architectural design that aligns closely with the principles of the 3-tier architecture. In the MVC pattern, the Model represents the underlying data and business logic, corresponding to the database server and the DAOs of the application server. The View represents the user interface, similar to the web client tier, and is responsible for presenting data to the user. Finally, the Controller, implemented using the Handlers shown above, serves as the intermediary, handling user input and updating the Model or View as necessary. This separation of concerns allows for modular design, facilitating maintainability, scalability, and ease of testing, making the MVC pattern a natural fit for modern web and software applications.

2.6.3 RESTful API

The communication between client and server is handled through a REST API. This way, queries are HTTP based and can have an additional level of security using SSL encryption with HTTPS. Moreover REST has caching capabilities, enhancing performance even at scale since requests can be resolved directly by the client. Another advantage over other communication styles is statelessness which leads to naturally independent requests that can be handled concurrently.

2.7 Other Design Decisions

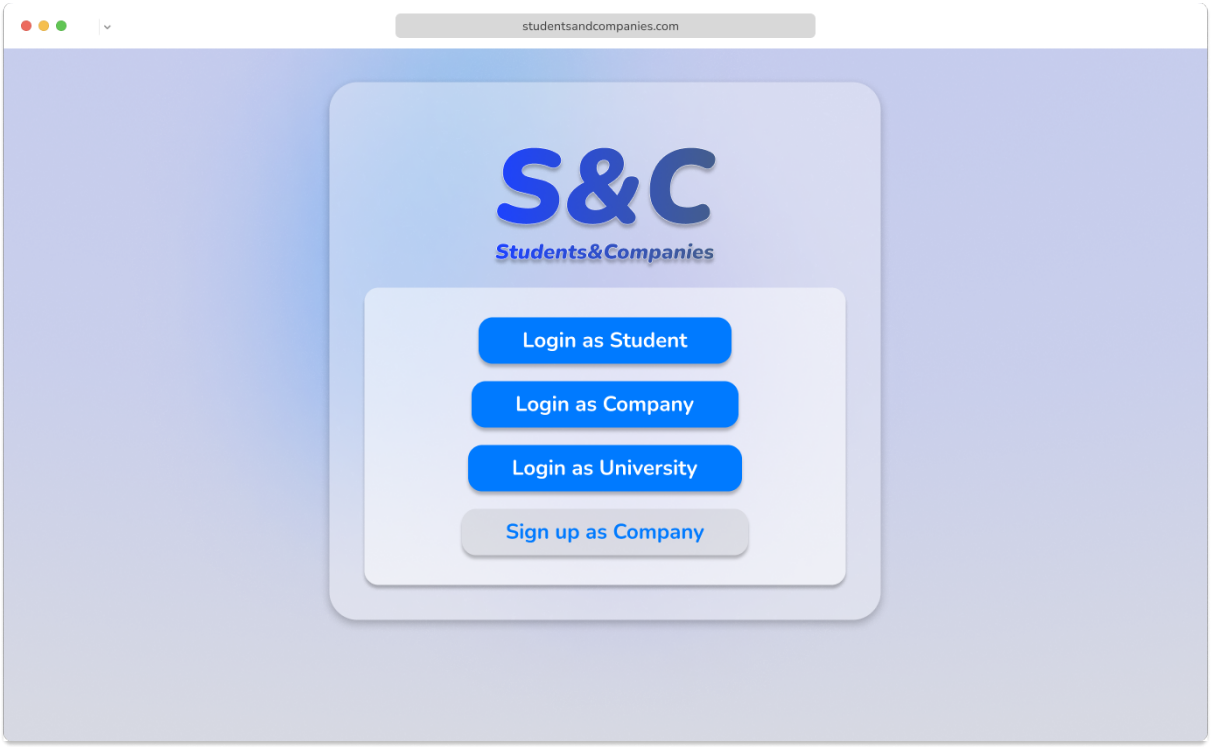
2.7.1 Load Balancer

A fundamental component in the system's architecture is the load balancer. This device distributes incoming requests between the replicated application servers to avoid overloads resulting in increased response times and to seamlessly add (or remove) servers as needed. Load balancers can also protect the system from DDOS attacks, rate limiting clients issuing requests too frequently.

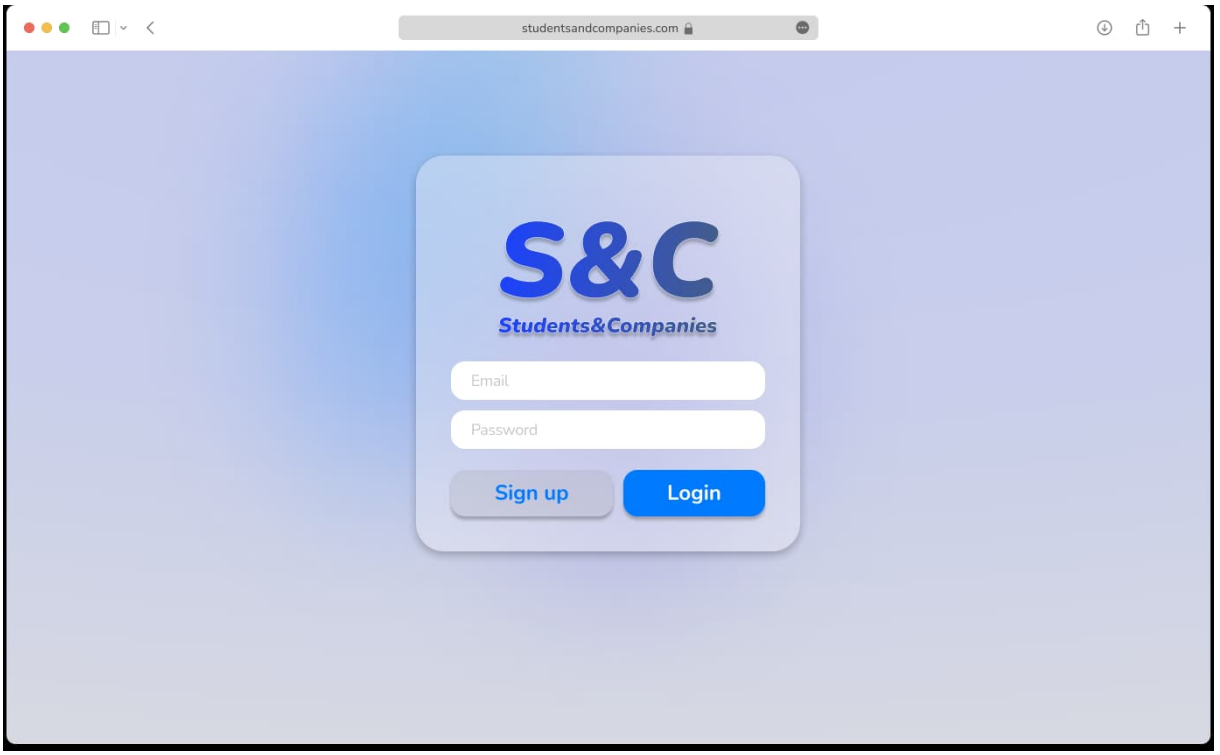
Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

3 User Interface Design

Initial Page

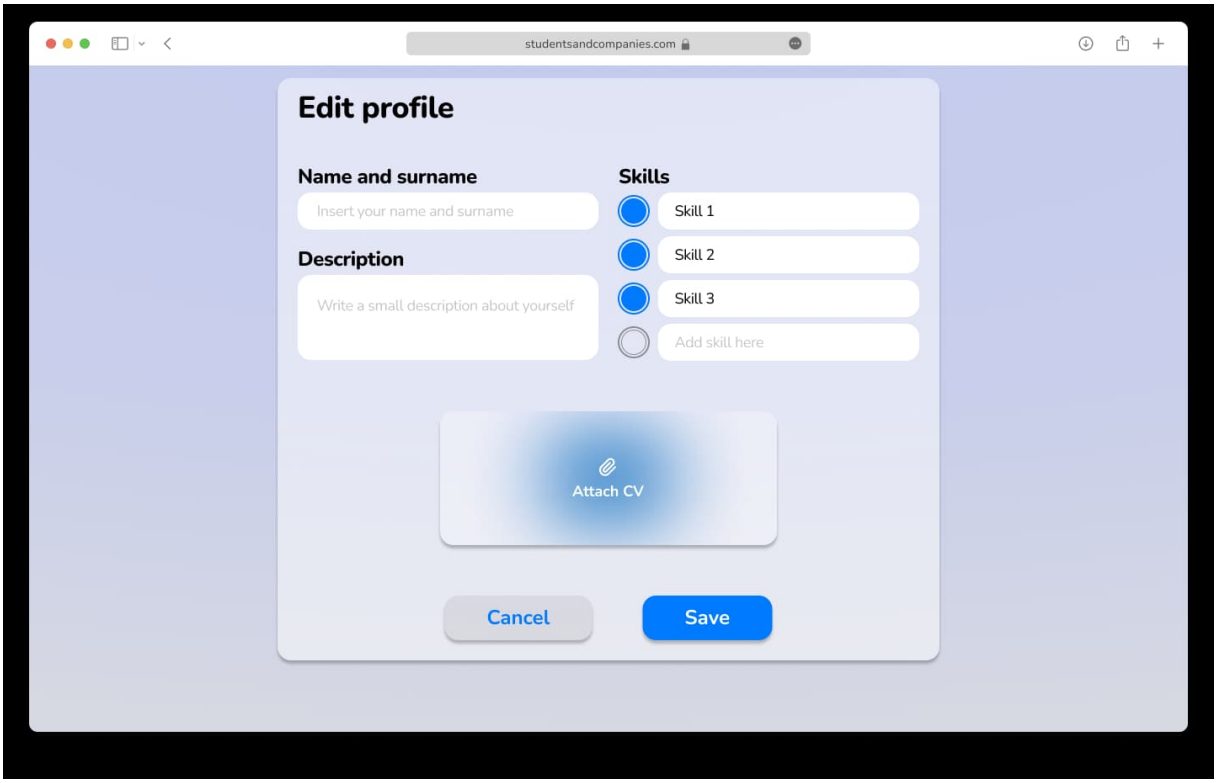


User Login

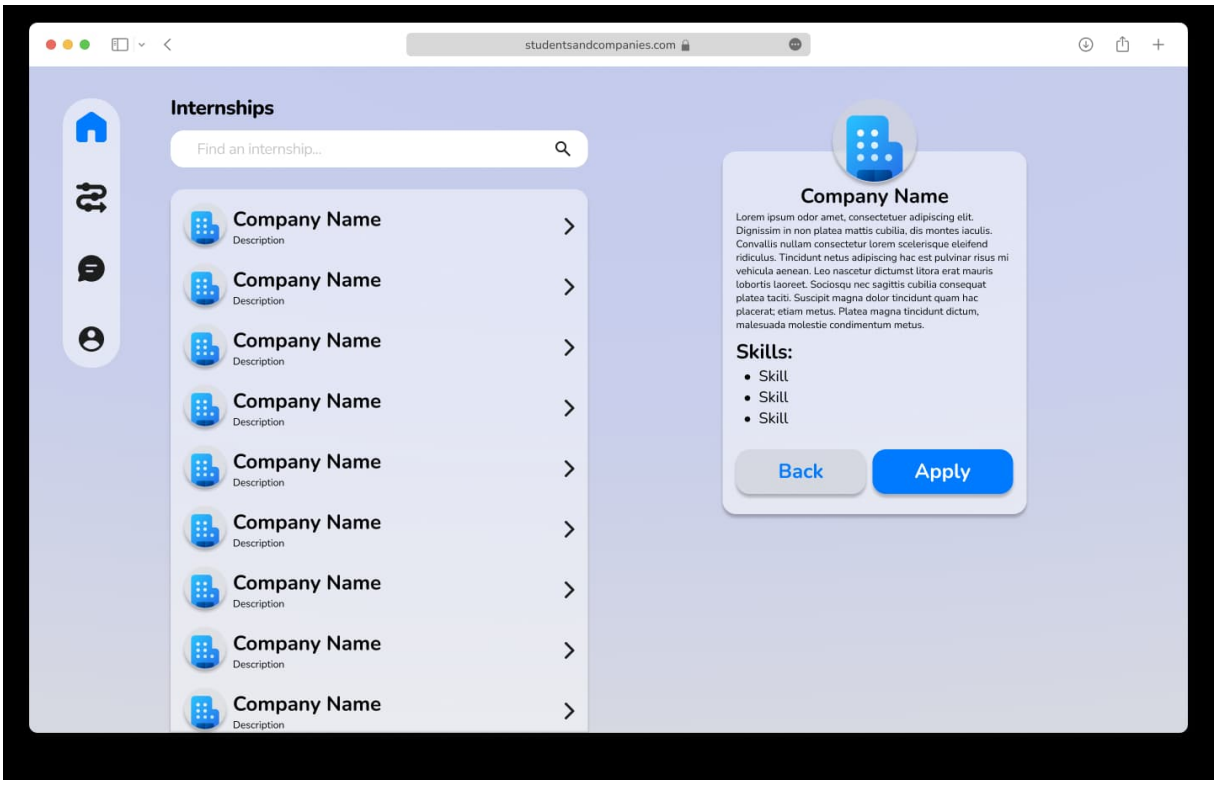


Edit Profile

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

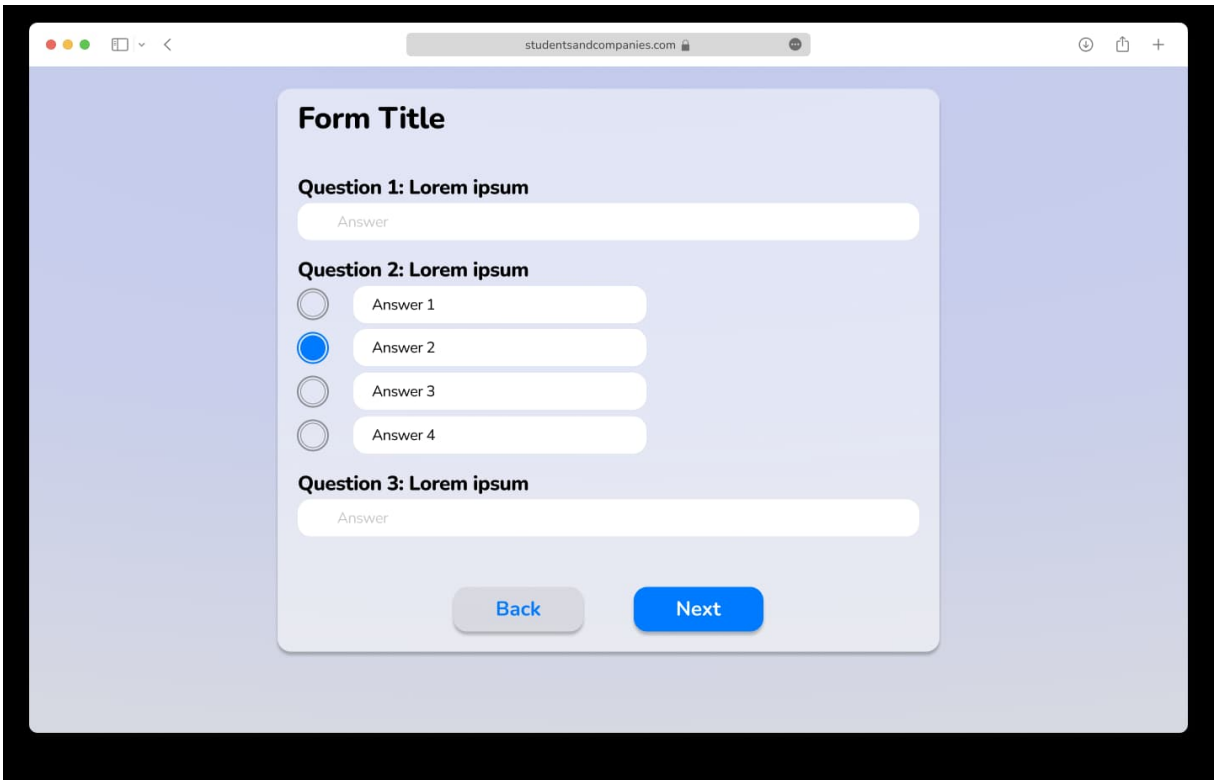


Search Internship

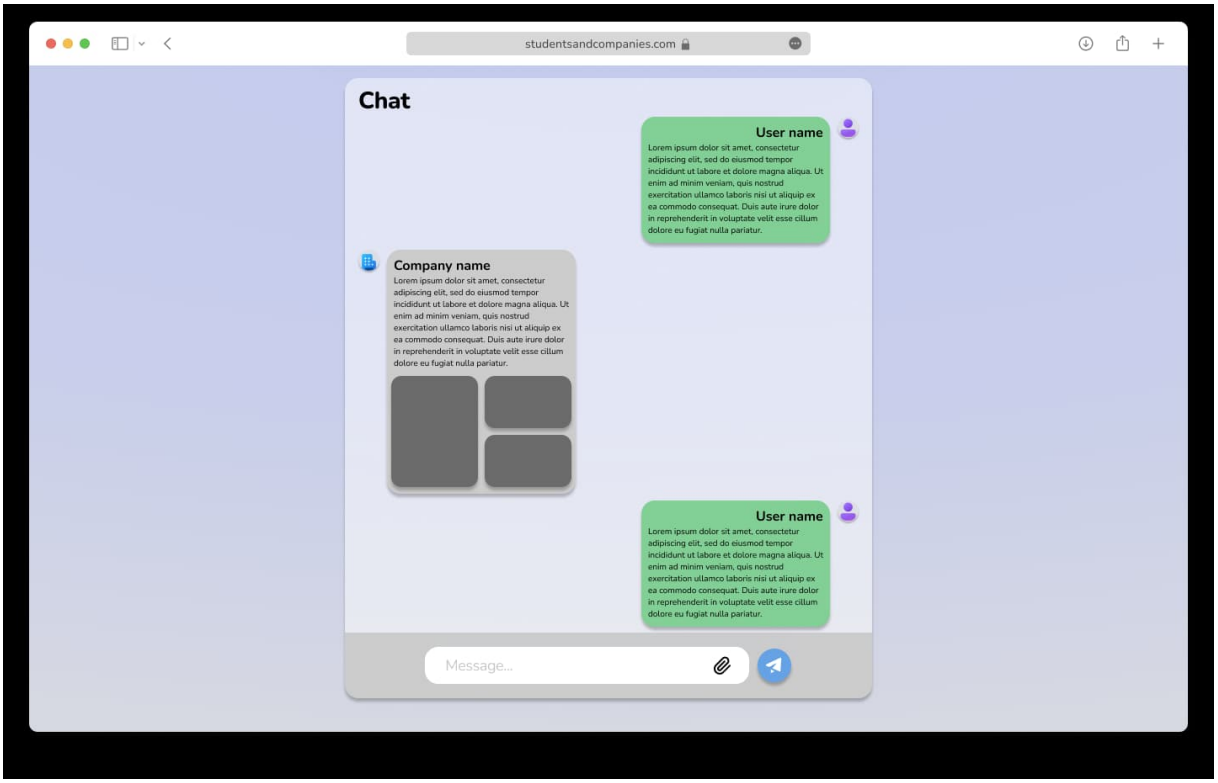


Create Questionnaire

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

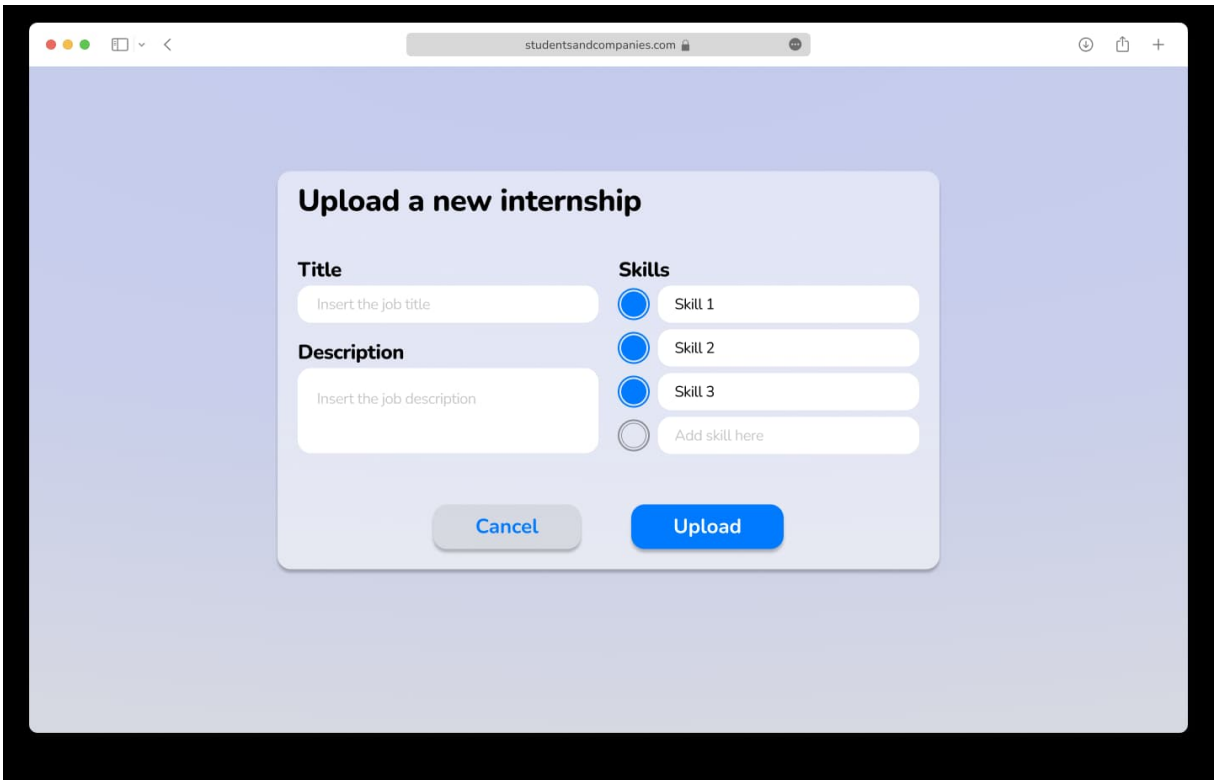


Discussion

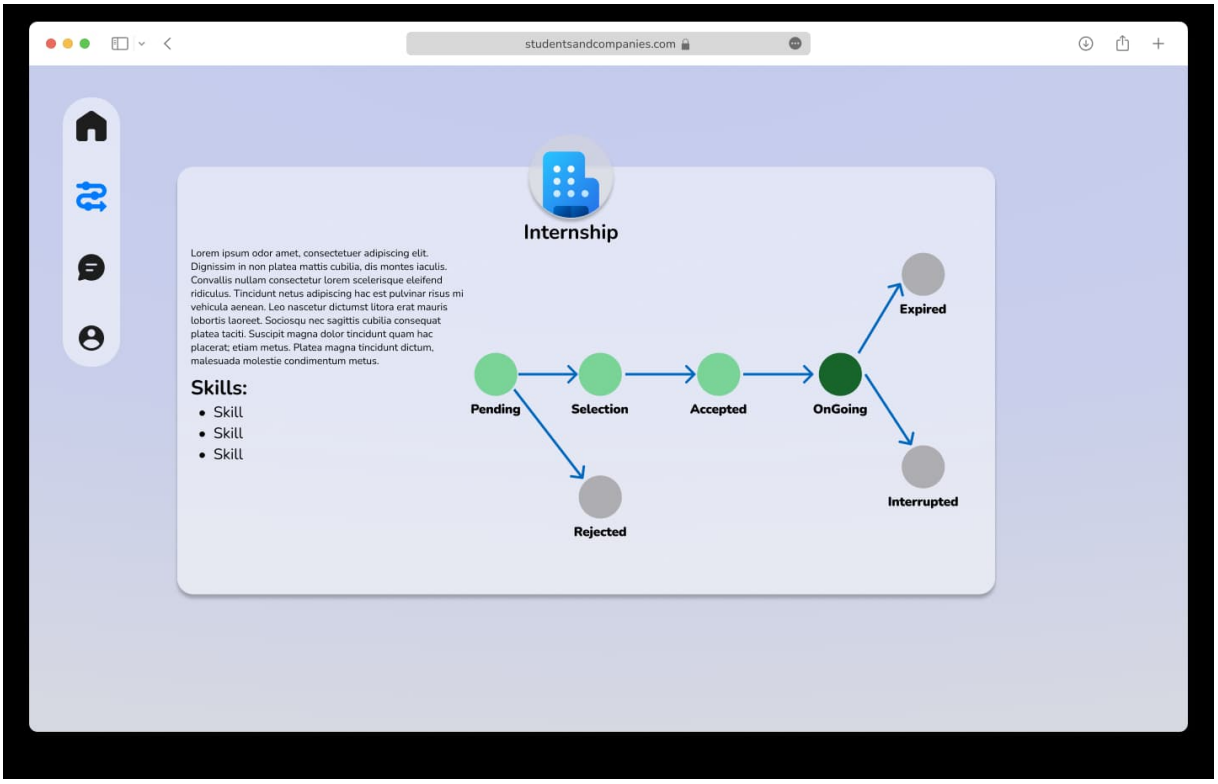


Upload Internship

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved



Monitor Status



Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

4 Requirements Traceability

Requirements	Login [R1] The system must allow students to log-in through their academic email and password, if they provide them correctly. [R2] Only students with an academic credential provided by a registered university can log-in.
Components	WebApp AuthenticationManager SheerId API DAO Database

Requirements	Edit profile [R3] The system must allow the student to edit his profile info, remove the CV and add a new version of the CV. [R4] The student can edit only his profile.
Components	WebApp AuthenticationManager AccountManager DAO Database

Requirements	Internship recommendation [R5] The system must send an email to the student when an internship that might interest him becomes available.
Components	Affinity Analyzer DAO Database Notification Manager Mail Service

Requirements	Search an internship [R6] The system must provide to the student all internships satisfying search criteria. [R7] The system must provide to the student only internships satisfying search criteria.
Components	WebApp Internship Manager DAO Database

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

Requirements	Apply for an internship [R8] The system must register the student's application for the internship. [R9] The system must register only applications of students who have submitted an application for the internship.
Components	WebApp Internship Manager DAO Database

Requirements	Answer interview questions [R10] The student must be able to answer interview questionnaires. [R11] The student can answer interview questionnaires regarding only internships he's applied for and accepted by the involved company.
Components	WebApp Contract Manager Interview Manager DAO Database

Requirements	Consulting CV suggestion [R12] The student must be able to view suggestions on how to improve his CV. [R13] The student can view suggestions regarding only his CV.
Components	CV Analyzer Notification Manager Mail Service DAO Database

Requirements	Registration and login [R14] The system must allow companies to register to S&C providing mandatory data (name, email, VAT number) and selecting a password. [R15] The system must allow registered companies to log-in through their email and password, if they are provided correctly.
Components	WebApp Authentication Manager DAO Database

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

Requirements	Upload an internship [R16] The system must allow the company to fill an internship creation form. [R17] The system must allow the company to select the internship type. [R18] The system must allow the company to select the internship duration. [R19] The system must allow the company to select if the internship is paid or not. [R20] The system must allow the company to select the skills it is searching for in the candidates.
Components	WebApp Internship Manager DAO Database

Requirements	Student recommendation [R21] The system must send an email notifying the company when a student with a CV corresponding to their needs becomes available.
Components	Mail Service Notification Manager Affinity Analyzer DAO Database

Requirements	Interview students [R22] The company must be able to provide structured questionnaires to students. [R23] The company can provide structured questionnaires only to students who have applied to an internship uploaded by the company itself. [R24] The company must be able to view answers to questionnaires it created.
Components	WebApp Contract Manager Interview Manager DAO Database

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

Requirements	Create contract [R25] The company must be able to register a contract regarding an internship and referring to the assigned student. [R26] The company can register contracts regarding only internships uploaded by the company itself. [R27] The company can register contracts referring only to students that have applied for the indicated internship.
Components	WebApp Contract Manager DAO Database

Requirements	Login [R28] The system must allow registered universities to log-in through credentials provided by S&C, if they are provided correctly.
Components	WebApp Authentication Manager DAO Database

Requirements	Terminate an internship [R29] The system must allow universities to terminate an internship involving their students.
Components	WebApp Contract Manager DAO Database

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

Requirements	Leave a feedback [R30] The student must be able to post feedback. [R31] The company must be able to post feedback. [R32] Only students that have participated in an internship can leave feedback for it. [R33] Only companies that have participated in an internship can leave feedback for it. [R34] Every student can leave at most one feedback per internship. [R35] Every company can leave at most one feedback per internship.
Components	WebApp Contract Manager Feedback Manager DAO Database

Requirements	Open discussion [R36] The student must be able to open a discussion with the company he's interning with and the university he's enrolled at. [R37] The company must be able to open a discussion with the students they are collaborating with and universities those students are enrolled at. [R38] The student can open a discussion only with the company he's interning with and the university he's enrolled at. [R39] The company can open a discussion only with students they are collaborating with and universities.
Components	WebApp Contract Manager Discussion Manager DAO Database

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

Requirements	Send and receive messages within a discussion [R40] The student can send and receive messages only from a discussion he is participating in. [R41] The company can send and receive messages only from a discussion it is participating in. [R42] The university can send and receive messages only from a discussion it is participating in.
Components	WebApp Contract Manager Discussion Manager DAO Database

Requirements	Monitor the status of the internship [R43] The student must be able to monitor the status of the internships in which he has applied. [R44] The company must be able to monitor the status of the internships it has published. [R45] The university must be able to monitor the status of the internship in which its students have applied.
Components	WebApp Contract Manager DAO Database

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

5 Implementation, Integration, and Test Plan

5.1 Overview

This section goes over how the S&C platform will be implemented and tested. Through testing it is possible to identify and correct possible bugs proactively, greatly decreasing the risk of a user having problems while using Students&Companies. In the following sections it will be described how the system will be implemented and how its components will be tested.

5.2 Testing and Implementation Plan

The system will be tested and implemented using a bottom-up approach, incrementally adding components as their dependencies are fully tested and integrated. This means that every component will be using its actual dependencies and will be activated by drivers that simulate the behaviour of the upstream components. This testing flow has the advantage of being straightforward and easy to adopt since it reduces the number of mock components that must be created and many components have the same dependencies hence they can be tested simultaneously, speeding up the implementation process.

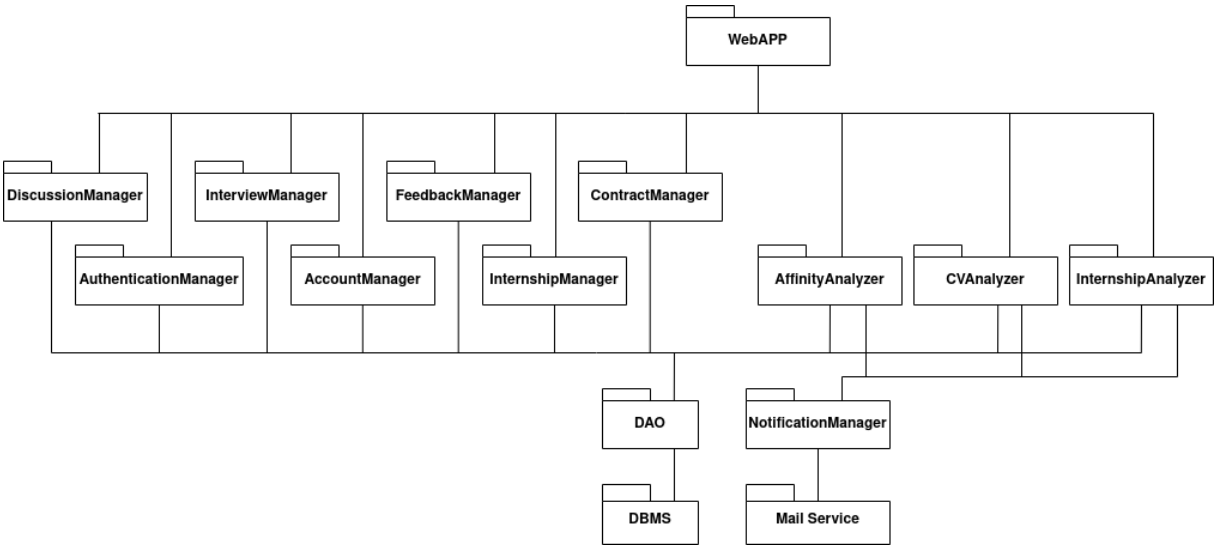


Figure 4: Dependencies of the components

In figure 4 are represented by the dependencies between components. Using the bottom-up approach means that for example, the analyzers will be tested after the testing of DAO and NotificationManager is completed. To test the WebAPP component the REST API will be used to submit requests and check that the responses are correct and no errors occur.

5.3 Additional System Testing

After the system is fully implemented, there are other aspects which can be tested to ensure that the platform is able to guarantee non-functional requirements such as availability, security and response times. To check this aspects the following tests can be done:

- **Performance testing**: a basic test in which requests are submitted simulating the expected load. This can be used to ensure that everything works as expected under normal conditions, resulting in a positive user experience, and to identify possible bottlenecks or inefficiencies.

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

- **Stress testing**: there is a possibility of the system receiving more requests than expected. By generating an artificially high number of requests it will be tested if the system is able to handle them or, in the eventuality of crashes due to the high load, is able to gracefully recover.
- **Security testing**: this test is aimed at finding possible vulnerabilities on the platform. This is performed by checking that malicious requests are successfully rejected.

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

6 Effort Spent

Bogdanovic Stefan

Section	Effort spent
1	1
2	14
3	2
4	8
5	1

Rodari Simone

Section	Effort spent
1	1
2	16
3	8
4	2
5	1

Pignataro Gabriele

Section	Effort spent
1	1
2	12
3	2
4	3
5	6

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved

References

[1] SheerID Inc. SheerID, 2024. URL: https://www.sheerid.com.

Copyright © 2024, Bogdanovic Stefan, Rodari Simone, Pignataro Gabriele – All rights reserved